



AI-Enhanced OCR for Medical Prescription Interpretation

By

Muhammad Tayyab

(ECI-IT-21-062)

Muhammad Shafqat

(ECI-IT-21-060)

Muhammad Usama

(ECI-IT-21-066)

Danial Afridi

(ECI-IT-20-063)

2025

Faculty of Engineering Sciences and Technology

Hamdard University Islamabad Campus, Pakistan



AI-Enhanced OCR for Medical Prescription Interpretation

By

Muhammad Tayyab

(ECI-IT-21-062)

Muhammad Shafqat

(ECI-IT-21-060)

Muhammad Usama

(ECI-IT-21-066)

Danial Afridi

(ECI-IT-20-063)

Under the supervision of

Dr. Shaheer Muhammad

2025

Faculty of Engineering Sciences and Technology

Hamdard University Islamabad Campus, Pakistan

AI-Enhanced OCR for Medical Prescription Interpretation

By

Muhammad Tayyab

(ECI-IT-21-062)

Muhammad Shafqat

(ECI-IT-21-060)

Muhammad Usama

(ECI-IT-21-066)

Danial Afridi

(ECI-IT-20-063)

A project presented to the
Department Of Computing

In partial fulfillment of the
requirements for the degree of

Bachelors of Science

In

Computer Science

Faculty of Engineering Sciences and Technology

Hamdard University Islamabad Campus, Pakistan



Faculty of Engineering Sciences and Technology
Hamdard University Islamabad Campus, Pakistan

CERTIFICATE

This project “AI-Enhanced OCR for Medical Prescription Interpretation” presented by **Muhammad Tayyab, Muhammad Shafqat, Muhammad Usama, and Danial Afridi** under the direction of their project advisor, **Dr. Shaheer Muhammad**, and approved by the project examination committee, has been presented to and accepted by the Faculty of Engineering Sciences and Technology, in partial fulfillment of the requirements for **Bachelor of Science in Computer Science**.

Dr. Shaheer Muhammad
(Project Advisor)

Dr. Tahir Saleem
(Examiner 1)

Dr. Muzammil Malik
(Examiner 2)

Dr. Tahir Saleem
(Chairman, Department of Computing)

Dr. Hassan Raza
(Associate Dean, FEST)

Abstract

Handwritten prescriptions present a persistent challenge in healthcare due to their illegibility, variations in writing styles, and lack of standardized formatting. These issues are particularly relevant in countries where electronic health record (EHR) adoption is still limited. While some OECD countries such as Denmark, Estonia, and Finland report near-universal use of digital prescription systems, handwritten prescriptions remain dominant in many developing nations, increasing the risk of misinterpretation of drug names. Therefore, this work introduces a pipeline-based OCR system tailored for handwritten medical prescriptions. The process begins with segmentation, where text regions are extracted and individual characters are isolated from prescription images. Character recognition is then carried out using deep learning models such as EfficientNet B0, B1, B2, and ResNet, which are designed to capture subtle variations in strokes, curves, and shapes of handwritten characters while balancing computational efficiency. This modular design ensures that the system can learn fine-grained handwriting features without relying on overly complex architectures.

Acknowledgement

All praises and thanks to Al-Mighty “ALLAH”, the Most Merciful, the Most Gracious, the source of knowledge and wisdom endowed to mankind, who conferred us with the power of mind and capability to take this project to the exciting ocean of knowledge. All respects are for our most beloved Holy Prophet “Hazrat MUHAMMAD (Peace Be Upon Him)”, whose personality will always be a source of guidance for humanity.

Acknowledgement is due to Hamdard Institute of Engineering and Technology for the support of this project, a highly appreciated achievement for us at the undergraduate level.

We wish to express our appreciation to our advisor, **Dr. Shaheer Muhammad**, for his keen guidance, sincere help, and friendly manner which inspired us to perform well in the project and made it a reality.

Many people, especially our classmates and team members themselves, have made valuable comments and suggestions on this project which gave us the inspiration to improve it further.

Finally, we would like to thank our family and friends for their moral support and encouragement throughout the project.

Project in Brief

Project Title	AI-Enhanced OCR for Medical Prescription Interpretation
Objective	To develop an AI-powered Optical Character Recognition (OCR) system capable of digitizing handwritten medical prescriptions and extracting pharmaceutical names to improve patient safety.
Undertaken By	• Muhammad Tayyab (ECI-IT-21-062) • Muhammad Shafqat (ECI-IT-21-060) • Muhammad Usama (ECI-IT-21-066) • Danial Afridi (ECI-IT-20-063)
Supervised By	Dr. Shaheer Muhammad
Duration of Project	• Start Date: October 2024 • End Date: September 2025
Tools & Technologies Used	• Programming Languages: Python • Frameworks & Libraries: TensorFlow, Keras, OpenCV, Django • Development Tools: Jupyter Notebook, Visual Studio Code • Hardware: Standard CPU-based system used for training and testing the AI models
System Used	Windows-based development system utilizing CPU processing for model training and testing.

Contents

Abstract	i
Acknowledgement	ii
Introduction	2
1.1 Motivation	2
1.2 Problem Statement	2
1.3 Organization of the Report	3
Literature Review	5
2.1 Existing Systems and Methods	5
2.2 Handwriting Recognition in Medical Prescriptions	5
2.3 Background and Related Work	6
2.4 OCR Techniques for Handwritten Text	7
2.5 AI and Deep Learning in OCR	7
2.6 Gaps in Existing Research	8
2.7 Summary	8
Methodology	11
3.1 Data Collection and Preprocessing	11
3.2 Data Sources	11
3.3 Importance of Data Collection and Preprocessing	14
3.4 Segmentation	14
3.5 Character Segmentation	15
3.6 Segmentation Fixes and Enhancements	16
3.7 Importance of Segmentation	16
3.8 Model Development	16

3.8.1	Comparison of Architectures: VGG, ResNet, and EfficientNet	16
3.9	Baseline Models for Character Recognition	17
3.9.1	Fine-Tuning Strategy	18
3.10	System Testing and Performance Evaluation	20
3.10.1	Testing Setup	21
3.10.2	Evaluation Metrics	21
3.10.3	Analysis of Test Results	22
3.10.4	Significance of Evaluation	22
3.11	Integration into Application	22
3.11.1	Workflow of the Application	23
3.11.2	Key Features of the Application	24
3.11.3	Significance of Integration	24
System Design		27
4.1	System Architecture	27
4.2	Functional Requirements	27
4.3	Non-Functional Requirements	28
4.4	Tools and Technologies Used	28
4.5	Application Flowcharts and Diagrams	29
4.6	User Interface Design	29
Results and Discussion		31
5.1	Experimental Setup	31
5.2	Performance Analysis	32
5.2.1	Segmentation of “Axodin”	34
5.2.2	Segmentation of “Atnizin”	34
5.2.3	Segmentation of “Aceta”	35
5.2.4	Segmentation of “Azithrocin” and Recognition Results	36
5.2.5	Segmentation of “Aceta” and Recognition Results	37
5.3	Character-Level Recognition Results	38
5.3.1	Character-Level Predictions	39
5.4	Model Training and Performance Analysis	40
5.4.1	Model Variants Evaluated	40

5.4.2	Training Strategy for EfficientNet-B0	41
5.4.3	Training Strategy for EfficientNet-B1, B2, and ResNet-18-Lite	41
5.4.4	Phase 1: Initial Training of EfficientNet-B0	42
5.4.5	Phase 2: Fine-Tuning with Lower Learning Rate	44
5.4.6	Phase 3: Extended Fine-Tuning with Checkpointing	46
5.4.7	Performance Discussion	47
5.4.8	Final Model Performance	47
5.5	Challenges Faced and Solutions Applied	47
5.6	Summary of Findings	48
Future Recommendations		50
6.1	Conclusion	50
6.2	Recommendations	50
6.3	Future Enhancements	51
Appendix A: Dataset Details		56
A.1	Dataset Overview	56
A.1.1	Data Sources	56
A.1.2	Preprocessing	56
A.1.3	Sample Images of Raw Prescriptions	57
A.1.4	Sample Images of Segmented Characters	57
APPENDIX B: Models and Django Code		59
B.1	Model Training and Implementation	59
B.1.1	EfficientNet-B0 Training Code	59
B.1.2	Django Backend Code	69
APPENDIX C: Alignment with Sustainable Development Goals (SDGs)		72

List of Figures

3.1	Preprocessing Pipeline.	13
3.2	Successful Segmentation: Clear character boundaries.	14
3.3	Architecture of EfficientNetB0.	19
3.4	Overall workflow of the proposed system.	23
4.1	Methodology of Proposed System.	29
5.1	Performance comparison of all models using standard OCR metrics.	33
5.2	Segmentation output for the handwritten word “Axodin,” showing six distinct character segments.	34
5.3	Segmentation output for the handwritten word “Atnizin” where each character was accurately identified, leading to a correct final prediction.	35
5.4	Segmentation and recognition of “Aceta.” Both modules produced the correct output, showing a successful case of accurate processing.	35
5.5	GUI segmentation output for “Azithrocin,” showing successful character isolation and visualization within the web interface.	36
5.6	GUI segmentation output for “Aceta,” showing accurate character isolation and visualization within the web interface.	37
5.7	Character-level prediction for “A.” The model correctly classified the input with a confidence score above 97%.	39
5.8	Character-level prediction for “d.” The model achieved correct recognition despite handwriting variability.	39
5.9	Character-level prediction for “c.” Demonstrating robustness against circular handwriting variations.	40
5.10	Training vs Validation Accuracy (Phase 1).	42
5.11	Training vs Validation Loss (Phase 1).	43
5.12	Training vs Validation Accuracy (Phase 2).	44
5.13	Training vs Validation Loss (Phase 2).	45

5.14	Confusion matrix for Phase 3 of EfficientNet-B0. Most predictions lie on the diagonal; errors occur mainly between visually similar characters (O/Q, X/Y).	46
5.15	Final performance comparison across Phase 1, 2, and 3 for EfficientNet-B0.	46
A.1	Sample handwritten prescription images from the dataset, demonstrating the variety of handwriting styles and character appearances.	57
A.2	Examples of segmented characters from medicine names. Each segmented image corresponds to a labeled class used for training the OCR models.	57

List of Tables

3.1	Example of Dataset with Image-to-Character Mapping.	12
5.1	Performance Comparison of Deep Learning Models.	33
C.1	Alignment of the Proposed System with Sustainable Development Goals (SDGs).	72

CHAPTER 1

Introduction

1.1 Motivation

Doctor prescriptions are a fundamental communication tool in healthcare, acting as an official record of a physician's diagnosis and treatment plan. They contain crucial information such as medicine names, dosages, frequency, and usage instructions, which are directly tied to patient safety and recovery. However, a large proportion of prescriptions are still handwritten, and doctors' handwriting is often difficult to read due to variations in writing styles, abbreviations, and inconsistent formats. This legibility problem leads to medication errors, wrong dosages, treatment delays, and in some cases, severe health risks.

Advancements in Optical Character Recognition (OCR), especially when combined with Artificial Intelligence (AI) and Deep Learning, provide a transformative opportunity to address these issues. Modern AI-powered OCR systems, using architectures such as Convolutional Neural Networks (CNNs) have demonstrated high accuracy in recognizing even complex handwriting. In healthcare, these systems can automatically extract critical prescription data, reduce human interpretation errors [1, 2], and integrate seamlessly into Electronic Health Records (EHRs). This project focuses on developing a specialized OCR system for pharmaceutical terms, ensuring accurate recognition and categorization of medicines to ensure patient safety.

1.2 Problem Statement

Despite rapid progress in healthcare digitization, handwritten prescriptions remain the predominant medium for communicating treatment instructions in many clinical environments. These prescriptions frequently suffer from poor legibility, inconsistent formatting, and the extensive use of non-standard abbreviations. Such characteristics make them difficult to interpret, creating a risk of medication errors, treatment delays, and inefficiencies in pharmacy workflows.

From a technical standpoint, existing Optical Character Recognition (OCR) systems are primarily designed for printed text and are not well-suited to the complexities of handwritten medical prescriptions. Variability in handwriting styles, low-quality or noisy images, and the specialized vocabulary of pharmaceutical terminology present challenges that conventional OCR solutions fail to address.

This research aims to bridge this gap by developing a domain-specific OCR framework tailored for handwritten prescriptions. The proposed approach focuses on prescription-specific data collection, advanced preprocessing to handle handwriting irregularities, and recognition methods

adapted to pharmaceutical terms. By targeting the unique challenges of medical handwriting, the system seeks to reduce reliance on manual entry, improve the reliability of prescription interpretation, and ultimately enhance patient safety.

1.3 Organization of the Report

This Report is organized into six chapters. **Chapter 1** introduces the research background, problem statement, aims, objectives, significance, scope, and overall structure of the report.

Chapter 2 presents a comprehensive literature review, analyzing existing OCR techniques, AI applications in handwriting recognition, and their role in pharmaceutical prescription processing, while also highlighting the research gaps. **Chapter 3** describes the research methodology, covering data collection, preprocessing, segmentation, AI model development, training, evaluation, and ethical considerations. **Chapter 4** focuses on the system design and implementation, detailing the system architecture, functional and non-functional requirements, tools utilized, flowcharts, and user interface design. **Chapter 5** discusses the experimental results, performance analysis, comparisons with existing systems, challenges encountered, and interpretations of the findings. Finally, **Chapter 6** concludes the study with a summary of findings, conclusions, recommendations, and possible directions for future enhancements.

CHAPTER 2

Literature Review

2.1 Existing Systems and Methods

Over the past decade, Optical Character Recognition (OCR) systems have evolved from simple pattern-matching approaches to sophisticated AI-driven solutions capable of handling complex handwritten inputs. Traditional OCR relied on template matching and character segmentation, but such methods struggled with inconsistent handwriting styles, irregular layouts, and non-standard abbreviations often present in medical prescriptions [3].

Commercial OCR systems such as Tesseract and ABBYY FineReader achieve high accuracy for printed text but remain limited when processing variable handwritten prescriptions. Medical-focused OCR tools integrated into pharmacy software exist, yet they are often tailored to specific hospital formats and lack generalization [4].

The rise of deep learning has introduced hybrid architectures such as Convolutional Recurrent Neural Networks (CRNNs), which combine spatial feature extraction with sequential modeling. Studies by [5] demonstrated that CRNN-based models outperform traditional OCR methods in prescription recognition, achieving validation accuracies above 70%. However, these systems still face challenges with extremely poor handwriting and low-quality images, underscoring the need for domain-specific optimization.

2.2 Handwriting Recognition in Medical Prescriptions

Handwriting recognition in the medical domain introduces challenges that extend beyond standard OCR tasks. Medical prescriptions are often written in inconsistent formats, with a mix of uppercase and lowercase characters, abbreviations, and brand names [6, 7]. To address these challenges, our system implements a structured pipeline beginning with preprocessing, where noise reduction and normalization techniques are applied to enhance image quality and prepare data for subsequent stages.

Following preprocessing, character-wise segmentation is performed to isolate individual symbols from handwritten prescriptions. Each segmented character is then recognized using a deep learning model with strong feature extraction capabilities to handle variations in handwriting styles. By focusing on character-level recognition, the system ensures a solid foundation for accurate downstream processing [3, 4].

2.3 Background and Related Work

Doctor's prescription recognition is a significant challenge in pharmaceutical medicine recognition due to the inherent variability in doctors' handwriting styles. These inconsistencies can lead to misinterpretations and critical consequences for patient safety. Unfortunately, current OCR systems still struggle to accurately process handwritten medical prescriptions, making it difficult for healthcare professionals to rely on automated technology [20].

To address these challenges, Deep Convolutional Recurrent Neural Networks (CRNNs) have been employed. This hybrid model combines CNN layers for spatial feature extraction with LSTM layers for sequential dependency modeling. Early tests have achieved validation accuracy of around 72%, demonstrating CRNNs' effectiveness for prescription recognition [8]. Data augmentation and optimization techniques further enhance robustness, allowing the models to adapt to diverse handwriting styles [5].

The EMNIST dataset has served as a key resource for training, particularly in segmenting and processing individual characters [9]. Other methods, such as CNN with Error Correcting Output Codes (ECOC), have shown improvements in multi-class classification tasks, while CNN-BiLSTM with CTC loss has enabled alignment-free sequence recognition, especially when combined with curated medical corpora and string-matching techniques [10, 11].

Several domain-specific systems also exist. For example, Dhande et al. (2024) proposed a model for cursive English handwriting using projection methods and convex hull algorithms, achieving 85% accuracy [12]. Hybrid approaches such as CNN with SVM have also achieved notable results, including 98.85% accuracy for numerals and 93.05% for uppercase characters on the NIST dataset [13]. Similarly, OCR-Nets optimized versions of AlexNet and GoogleNet achieved 96.3% accuracy for Urdu characters using transfer learning [14].

Other studies have explored machine learning-based prescription recognition using preprocessing (noise reduction, normalization) and CNN-based classification, reporting training accuracies of 73% and testing accuracies of 50% [15]. Meanwhile, RNNs with CTC loss [16] and CNNs trained on the NIST Special 19 Database [17] have also shown strong performance when supported by preprocessing steps such as noise removal and segmentation.

Recent advances, including CRNNs and Faster R-CNN, have achieved superior accuracy in both character segmentation and recognition, making them promising candidates for real-world deployment in healthcare environments [5, 18].

In contrast to many of these end-to-end recognition methods, our work adopts a structured pipeline tailored specifically for handwritten medical prescriptions. The process begins with preprocessing to enhance image clarity and reduce noise, followed by character-wise segmentation to isolate individual symbols. Each segmented character is then recognized using an EfficientNet-based model, which provides strong feature extraction for handling diverse handwriting styles.

2.4 OCR Techniques for Handwritten Text

Handwritten OCR techniques can broadly be divided into rule-based and machine learning-based approaches. Rule-based methods depend on predefined stroke patterns and character templates, but are brittle against handwriting variability. In contrast, machine learning-based methods learn directly from data, enabling adaptability and improved accuracy [3, 4]. Key advancements include:

- **Convolutional Neural Networks (CNNs):** Effective for spatial feature extraction and distinguishing character shapes.
- **Recurrent Neural Networks (RNNs):** Capture sequential dependencies, improving recognition of connected handwriting.
- **Hybrid CNN-RNN (CRNNs):** Combine spatial and sequential learning for higher accuracy in sequence-based handwriting tasks.
- **Transfer Learning:** Leveraging pre-trained models such as AlexNet and GoogleNet for OCR, as demonstrated by Aarif and Poruran (2020), achieving up to 96.3% accuracy in Urdu character recognition.

Image preprocessing techniques such as noise reduction, grayscale conversion, and binarization remain essential for improving recognition performance.

2.5 AI and Deep Learning in OCR

AI and deep learning have transformed OCR, particularly in handling complex handwritten scripts. CNNs excel in extracting high-level features, while advanced CNN layers capture contextual information from both preceding and following characters [10, 14]. In the medical domain, deep learning models are often integrated with medical terminology databases to

validate recognized text and correct errors. This is critical since prescription OCR must ensure that recognized terms align with approved drug names [11, 15]. Recent research trends include data augmentation for model robustness, lightweight neural networks for real-time OCR, and cloud-based solutions for scalability [5, 8]. However, deploying these systems in healthcare requires compliance with privacy standards such as HIPAA and GDPR [1, 19].

2.6 Gaps in Existing Research

Despite significant progress, prescription-specific OCR still faces key challenges:

1. **Domain-Specific Optimization:** Generic OCR tools are not trained on pharmaceutical terminology, causing frequent misclassification of drug names and abbreviations.
2. **Handwriting Variability:** High inter-doctor variability, cursive writing, and inconsistent formats remain major obstacles for reliable recognition.
3. **Character-Level Recognition:** Existing models often struggle with segmentation errors and overlapping strokes, reducing accuracy for individual characters.
4. **Lack of Complete Pipelines:** Many studies emphasize raw recognition accuracy but overlook the importance of a structured pipeline that includes preprocessing, segmentation, character recognition, and post-processing.
5. **Word Reconstruction:** Few approaches incorporate post-recognition correction or word reconstruction algorithms to handle incomplete or misclassified sequences and accurately reconstruct full medicine names.
6. **Practical Deployment:** Limited work has explored integration of prescription OCR into real-world hospital and pharmacy systems for reliable, real-time usage.

2.7 Summary

The literature reveals major advancements in OCR, particularly through CNNs, RNNs, CRNN-based models, and hybrid approaches that combine deep learning with preprocessing or string-matching techniques. These methods have significantly improved handwritten text recognition and are increasingly applied to medical prescriptions [5, 11, 14, 16].

However, persistent challenges remain—including extreme handwriting variability, poor image quality, and complex pharmaceutical terminology [15, 20]. The scarcity of annotated prescription datasets and limited deployment of OCR in real-world healthcare systems highlight a research gap [9, 21].

This opens opportunities for developing domain-specific, AI-powered OCR systems optimized for medical prescriptions. Such systems, integrating advanced machine learning models with medical terminology validation and robust preprocessing pipelines, have the potential to enhance patient safety, reduce manual workload, and streamline healthcare workflows [1, 2].

CHAPTER 3

Methodology

3.1 Data Collection and Preprocessing

The foundation of any Artificial Intelligence (AI) or Machine Learning (ML) project lies in the quality and diversity of its dataset. For this OCR prescription recognition system, data collection and preprocessing were the most crucial steps in ensuring reliable training, testing, and deployment. This phase was designed to cover a wide variety of handwriting styles, prescription formats, and image conditions, making the model adaptable to real-world use cases.

3.2 Data Sources

The dataset was collected from the Kaggle platform, which provided a diverse set of handwritten medicine names along with their corresponding ground-truth text labels. This dataset was chosen because it already contained pharmaceutical terms, making it directly relevant to the project's domain.

- **Public Datasets (Kaggle):** The primary dataset was sourced from Kaggle [21], which provided handwritten images of medicine names along with ground-truth text labels. This dataset was chosen because it already contained pharmaceutical terms, making it directly relevant to the project's domain.

Dataset Preparation

We worked with a Kaggle dataset that contains images of handwritten medicine names. The preparation process is essential for training and fine-tuning our OCR models to improve recognition accuracy. Below is a detailed overview of the dataset preparation steps:

- **Dataset Source:** The dataset originates from Kaggle and contains images of handwritten medicine names.
- **Manual Annotation:** Each image is labeled to ensure high accuracy, with clear mappings between the image and the corresponding medicine name.
- **Structured Labeling:** For each image, the medicine name is broken down into individual characters. For example, "Aceta" is labeled as A, c, e, t, a, ensuring consistency across all images.

- **Character Image Extraction:** Using our custom segmentation pipeline, character-level images are extracted from each word. These are then saved and linked to labels (e.g., `image_label_1`, `image_label_2`, . . . , `image_label_5`) for precise mapping.
- **Improved Training Readiness:** This structured dataset allows for a direct connection between each character image and its ground truth label. This organization significantly enhances the quality and efficiency of training the OCR model.
- **Purpose:** This dataset preparation ensures the dataset is not just a collection of images but a high-quality, task-specific resource ready for effective model training and evaluation.

IMAGE	MEDICINE_NAME	char_1	char_2	char_3	char_4	char_5	image_label_1	image_label_2	image_label_3	image_label_4	image_label_5
0.png	Aceta	A	c	e	t	a	image_A	image_c	image_e	image_t	image_a
1.png	Aceta	A	c	e	t	a	image_A	image_c	image_e	image_t	image_a
2.png	Aceta	A	c	e	t	a	image_A	image_c	image_e	image_t	image_a
3.png	Aceta	A	c	e	t	a	image_A	image_c	image_e	image_t	image_a
4.png	Aceta	A	c	e	t	a	image_A	image_c	image_e	image_t	image_a
5.png	Aceta	A	c	e	t	a	image_A	image_c	image_e	image_t	image_a

Table 3.1: Example of Dataset with Image-to-Character Mapping.

Preprocessing Techniques

To prepare medicine name images for segmentation and recognition, a preprocessing pipeline was implemented:

- **Resizing:** Each image is resized to 128×128 pixels (or fixed width with preserved aspect ratio) to ensure uniformity across the dataset. This improves stability during model training and speeds up convergence.
- **Grayscale Conversion:** Images are converted from RGB to grayscale (single channel), lowering computational cost and focusing only on intensity patterns.
- **Noise Reduction:** Bilateral filtering (or Gaussian Blur for handwritten cases) is applied to reduce background noise, scanning marks, and pen pressure artifacts while preserving edges.
- **Contrast Enhancement:** CLAHE (Contrast Limited Adaptive Histogram Equalization) is applied to enhance visibility of faint handwriting strokes and balance uneven illumination.

- **Thresholding:** Otsu's adaptive method is applied to separate text (foreground) from the page background, providing Resizing: Maintain aspect ratio with a fixed width to Normalize input. Grayscale Conversion: Simplify image for thresholding and binary operations. Noise Reduction: Apply bilateral filtering to preserve edges and reduce noise. Contrast Enhancement: Use CLAHE (Contrast Limited Adaptive Histogram Equalization) on the L-channel of the LAB color space. Different Thresholding: Smart Binarization handling. Morphological Cleaning: Perform morphological opening to remove small noises. explain each step in detail binarization even under uneven illumination. For prescriptions with inverted ink/background, the polarity is automatically corrected [22, 23].
- **Morphological Cleaning:** Operations such as erosion, dilation, and opening are applied to reduce small noise and reconnect broken strokes [3, 23].

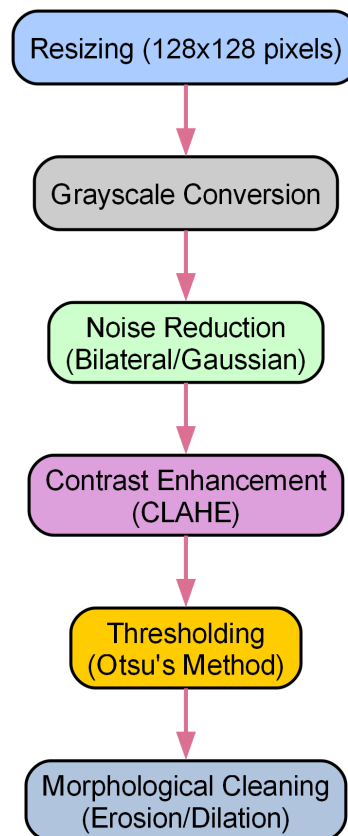


Figure 3.1: Preprocessing Pipeline.

Character Segmentation

A character segmentation pipeline was developed to isolate individual characters from medicine name images. The segmentation process consists of the following stages:

- **Projection-Based Analysis:** Vertical projection histograms identify gaps between characters, which guide the separation of individual regions.
- **Thresholding and Morphological Operations:** After Otsu's binarization, morphological cleaning is applied to emphasize text regions and suppress non-text noise.
- **Segment Validation:** Very narrow or excessively small regions are discarded. If expected segments are missing, dummy black images are inserted to maintain alignment with labels.
- **Saving and Normalization:** Each character segment is resized to 128×128 pixels and normalized before storage for recognition training.

The following figures illustrate correct segmentation and common segmentation challenges:

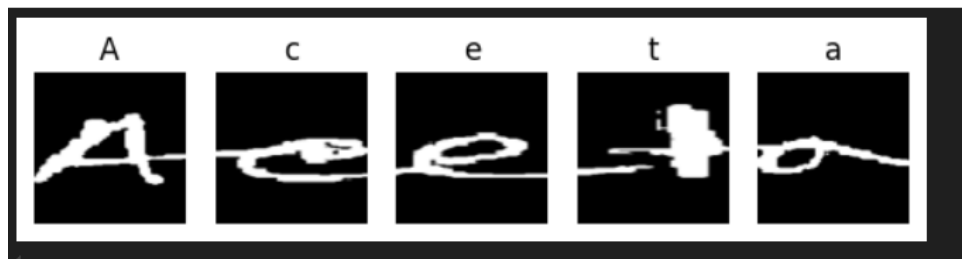


Figure 3.2: Successful Segmentation: Clear character boundaries.

3.3 Importance of Data Collection and Preprocessing

- **Variability Capture:** Collecting from multiple sources ensured wide handwriting style coverage.
- **Noise Handling:** Preprocessing reduced distractions from background artifacts, focusing the model on relevant features.
- **Scalability:** Normalized dataset allowed for smooth integration into deep learning pipelines.

3.4 Segmentation

Segmentation is a critical step in the OCR pipeline, as it isolates meaningful regions in prescription images primarily medicine names. Accurate segmentation directly affects recognition accuracy, since errors at this stage can propagate into misclassification during the recognition

phase. The segmentation process in this project combined projection-based methods with corrective strategies, while also evaluating advanced deep learning alternatives for robustness.

3.5 Character Segmentation

After line detection, segmentation proceeded to isolate individual characters within each line.

- **Method Used:**

- A vertical projection profile was calculated, analyzing the density of black pixels in each column of the image.
- Peaks: Represent character strokes.
- Valleys: Represent spaces between characters.

- **Why it Matters:**

- OCR models trained at character-level granularity achieve higher accuracy, especially for unseen medicine names.
- Recognizing characters individually allows reconstruction of full words without requiring the model to have seen the exact medicine name before.

- **Challenges:**

- Letters like “rn” could be merged and misread as “m.”
- Characters like “i” could be split into multiple parts (dot + stem).
- Overlapping strokes in cursive writing often blurred boundaries.

- **Fixes Applied:**

- Morphological Merging: Joined broken parts of characters (e.g., dot on “i” with its stem).
- Width Thresholding: Prevented splitting a single character into multiple fragments by enforcing minimum width.
- Over-segmentation Handling: Small, closely spaced segments were regrouped if they belonged to the same character.

3.6 Segmentation Fixes and Enhancements

To handle prescription-specific challenges, additional strategies were applied:

1. **Merging Broken Strokes:** Ensured tiny detached elements (like “i” dots, “j” dots, or punctuation marks) were properly associated with their base characters.
2. **Handling Over-Segmentation:** When thin gaps within a character caused unnecessary splits, corrective merging rules were applied.
3. **Noise Thresholding:** Very small connected components (dust particles, paper spots) were ignored, reducing false segmentation.

These fixes significantly improved the quality of character-level datasets, providing cleaner inputs for training.

3.7 Importance of Segmentation

- **Reduces Noise:** Ensures irrelevant areas of prescriptions are ignored.
- **Improves Recognition:** Provides clean, isolated characters for Model Training.
- **Supports Generalization:** Character-level segmentation allows recognition of unfamiliar medicine names by reconstructing words.
- **Bridges Preprocessing and Recognition:** Acts as the transition between image cleaning (Section 3.1) and model training (Section 3.3).

3.8 Model Development

The development of the Optical Character Recognition (OCR) system was carried out in a structured, iterative manner. The primary objective was to design a model capable of handling the irregularities of handwritten medical prescriptions, such as inconsistent spacing, variable stroke thickness, and overlapping characters.

3.8.1 Comparison of Architectures: VGG, ResNet, and EfficientNet

To identify the most suitable model for prescription OCR, three widely used CNN architectures were considered: VGG, ResNet, and EfficientNet. Each architecture offers distinct design

principles and trade-offs that impact feature extraction quality, computational cost, and transferability.

VGGNet introduced the idea of deep networks with small (3×3) convolution filters, providing strong baseline performance for image classification [24]. However, VGG is known to be computationally expensive, with a large number of parameters (about 138 million for VGG16), which limits its suitability for real-time applications [4].

ResNet addressed the problem of vanishing gradients in very deep networks by introducing residual connections. This innovation allowed much deeper models to be trained effectively, improving accuracy across vision tasks. ResNet is more efficient than VGG, but its computational overhead remains significant when scaled to very deep variants [25]. In this work, ResNet was implemented to evaluate its performance alongside EfficientNet.

EfficientNet, in contrast, provides a compound scaling method that balances network depth, width, and resolution in a principled way [26]. This architecture achieves state-of-the-art accuracy with significantly fewer parameters and lower computational cost compared to both VGG and ResNet. Its efficient design, combined with strong feature extraction capabilities, makes it particularly suitable for prescription OCR, where high accuracy is critical. EfficientNet B0, B1, and B2 variants were used in this study for a comprehensive comparison.

Given these considerations, both ResNet and EfficientNet were evaluated as backbone architectures for the OCR system, combining strong feature extraction capabilities with computational efficiency. Previous studies in prescription OCR also highlight the effectiveness of deep CNNs for handwritten recognition tasks, further supporting this choice [11, 15].

3.9 Baseline Models for Character Recognition

Before deploying a full-scale OCR system, multiple state-of-the-art convolutional architectures were implemented in TensorFlow to evaluate their ability to classify segmented characters from handwritten prescriptions (medicine names). These include:

- **EfficientNet** variants: B0, B1, and B2 [26], to explore trade-offs between accuracy, model size, and computational cost.
- **ResNet**: Deep residual networks [25], to assess performance with residual connections and deep feature extraction.

All models were trained on the same 52-class handwritten character dataset, using identical

preprocessing and segmentation pipelines, allowing a fair performance comparison.

- **Architecture Components:**

1. **Convolutional Layers:** Extract low- and mid-level features such as strokes, curves, and edges from handwritten characters.
2. **Scaling / Residual Mechanisms:**
 - EfficientNet: Compound scaling balances depth, width, and resolution [26].
 - ResNet: Residual connections enable training of deeper networks without vanishing gradients [25].
3. **Fully Connected Layers (Dense + ReLU):** Transform features into high-level representations suitable for classification.
4. **Output Layer (Softmax):** Produces probability distributions across the 52 character classes for final classification.

- **Purpose of the Baseline Model:**

- Established a baseline recognition pipeline for handwritten characters.
- Validated that the segmented character dataset was appropriate for training deep learning models.

- **Outcome:**

- Demonstrated the capability of EfficientNet to recognize isolated handwritten characters, leveraging its scalable architecture for effective feature extraction [26].
- Showed consistent improvements over alternative deep learning approaches reported in prescription OCR studies [11, 15].
- Confirmed the suitability of EfficientNet for character-level recognition, establishing it as the foundation for the final OCR pipeline [26].

3.9.1 Fine-Tuning Strategy

To adapt the convolutional architectures (EfficientNet variants B0, B1, B2, and ResNet) to the 52-class handwritten character dataset, a structured three-phase fine-tuning approach was

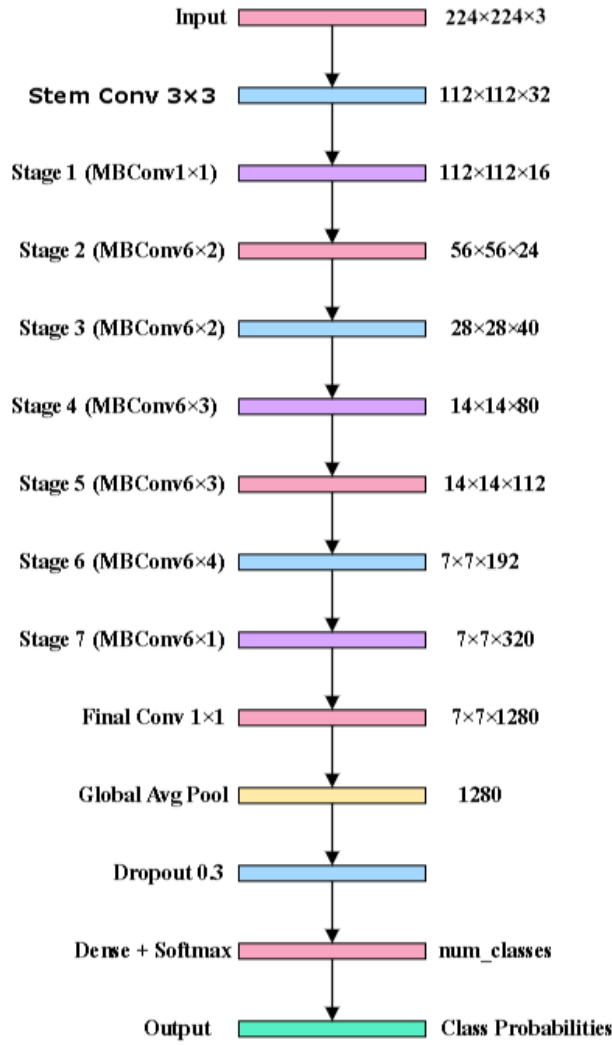


Figure 3.3: Architecture of EfficientNetB0.

adopted. This strategy enabled a gradual transition from generic features learned from large-scale datasets (e.g., ImageNet) to the specialized features required for handwritten medical prescriptions, while minimizing the risk of forgetting pretrained knowledge.

The learning rate (η) is a critical hyperparameter that controls the magnitude of weight updates during training. High values may cause divergence or oscillations, while very small values can slow convergence. For fine-tuning, a small learning rate (e.g., 1×10^{-5}) was chosen to allow subtle adjustments to pretrained features without overwriting them.

1. **Phase 1 – Base Frozen Training:** All backbone layers of the network were kept frozen, and only the newly added classifier layers (dense layers with softmax activation for 52 classes) were trained. This allowed the classifier to adapt existing feature maps to the

specific output classes without altering low-level features, ensuring stable convergence and reducing overfitting on a relatively small dataset.

2. **Phase 2 – Partial Fine-Tuning:** After the classifier stabilized, deeper layers were unfrozen for fine-tuning using a low learning rate. For EfficientNet, this included the last 50 convolutional layers; for ResNet, the last residual block was unfrozen. This phase enabled the models to refine intermediate feature representations to better capture handwriting nuances while preserving general low-level features.
3. **Phase 3 – Extensive Fine-Tuning:** In the final stage, additional layers were unfrozen for full adaptation: the last 150 layers for EfficientNet variants and the deeper residual blocks for ResNet. This allowed the network to progressively refine representations across multiple hierarchies, particularly improving distinction between visually similar characters (e.g., “c/o”, “i/l”, “n/u”). Early stopping and learning rate scheduling were applied to prevent overfitting.

This progressive fine-tuning strategy balances the retention of pretrained general features with learning highly specific patterns from handwritten prescription data, ensuring robust performance across diverse handwriting styles and image qualities.

Benefits: This gradual unfreezing strategy allowed stable adaptation, prevented forgetting the pretrained features, and maximized recognition accuracy for the isolated character dataset.

3.10 System Testing and Performance Evaluation

The OCR system tested by the test section of the dataset, which contained handwritten medicine names not seen during training. This evaluation phase was critical to assess the model’s ability to generalize to new, unseen prescriptions.

- Noisy or faint handwriting
- Overlapping or irregularly spaced characters
- Inconsistent prescription formats across different doctors or hospitals

By combining AI and Deep Learning techniques OCR system demonstrated a reliable ability to extract medicine names , making it a practical tool for healthcare digitization.

3.10.1 Testing Setup

The OCR system was evaluated using a portion of the collected dataset reserved for testing:

- **Dataset Split:**
 - 80% of the dataset was used for training.
 - 20% was used for testing to check the system's accuracy.
- **Evaluation:** The model was tested on the reserved test dataset to verify its performance in recognizing handwritten medicine names.

3.10.2 Evaluation Metrics

Multiple evaluation metrics were used to provide a comprehensive performance profile:

- **Accuracy:**
 - **Definition:** Percentage of correctly recognized drug names compared to the ground truth.
 - **Purpose:** Measures overall system correctness.
 - **Formula:**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad [27]. \quad (3.1)$$

- **Precision & Recall:**

- **Precision:** The ratio of correctly recognized drug names to all names identified by the model,

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (3.2)$$

- **Recall:** The ratio of correctly recognized drug names to the total actual drug names present in the dataset.

$$\text{Recall} = \frac{TP}{TP + FN} \quad [28]. \quad (3.3)$$

- **Purpose:** Ensures that the model is both exact (precision) and complete (recall).

- **F1-Score:**

- **Definition:** Harmonic mean of precision and recall,

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (3.4)$$

- **Equivalent (in terms of TP/FP/FN):**

$$F1 = \frac{2TP}{2TP + FP + FN} \quad [29]. \quad (3.5)$$

- **Purpose:** Provides a balanced view of the model's ability to avoid both false positives and false negatives.

3.10.3 Analysis of Test Results

- **Accuracy & F1-Score:** The EfficientNet recognized medicine names prescriptions.
- **Precision vs. Recall:**
 - Precision remained consistently high.
 - Recall was slightly lower for extremely poor-quality samples, where the model occasionally missed faint or broken characters.

3.10.4 Significance of Evaluation

The multi-metric evaluation confirmed that the OCR system was:

- **Reliable:** Recognizing drug names across different conditions.
- **Clinically Practical:** Integration with the terminology database ensured safe and valid outputs.

This stage provided the evidence base that the proposed OCR solution was not only theoretically sound for deployment in real-world healthcare environments.

3.11 Integration into Application

The final stage of the OCR system involved embedding the trained model into a user-friendly application tailored for healthcare professionals such as doctors, pharmacists, and hospital staff.

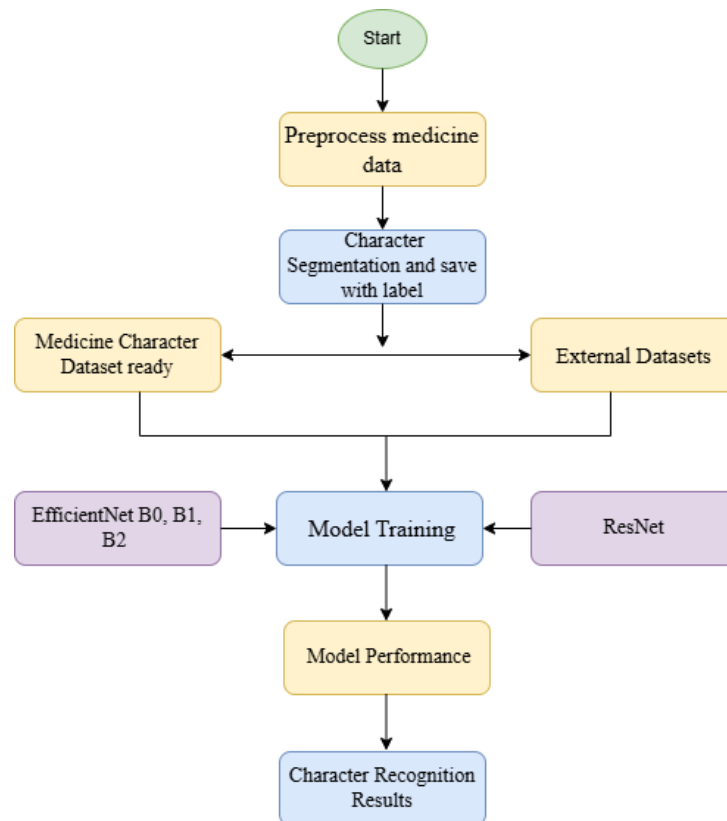


Figure 3.4: Overall workflow of the proposed system.

This ensured that the system moved beyond experimental testing and became a practical tool for real-world prescription handling.

3.11.1 Workflow of the Application

The application was designed around a simple, intuitive workflow to minimize training requirements for end-users:

1. Image Capture/Upload

- Users can upload prescription medicine names images via a web interface.

2. OCR Processing

- The integrated EfficientNet OCR model processes the image to extract medicine names
- Preprocessing (resizing, noise reduction, thresholding) is automatically applied in the background to improve recognition quality.

3. Output Display & Review

- Results are presented in a structured digital format.
- Pharmacists can review and edit entries before saving them into electronic health records (EHRs).
- The editable interface ensures human oversight remains part of the process.

3.11.2 Key Features of the Application

- **Smooth Navigation**

- Simple layout ensures users can process prescriptions with minimal clicks.

- **Lightweight Design for Healthcare Settings**

- Optimized to run on standard pharmacy computers, tablets, and mobile devices.
- Requires minimal hardware resources, making it deployable even in low-resource healthcare environments.

- **System Compatibility**

- Designed for easy integration with existing healthcare and pharmacy management systems.
- Export formats (CSV, JSON, or direct database entry) ensure interoperability with hospital record systems.

- **Privacy & Security Measures**

- **Encryption:** All prescription images and extracted text are encrypted during processing and storage.
- **Anonymization:** Patient-identifying information is removed where possible, ensuring compliance with healthcare privacy standards.
- **Regulatory Compliance:** The system follows guidelines like HIPAA and GDPR, ensuring sensitive medical data is protected.

3.11.3 Significance of Integration

The integration transformed the OCR model from a research prototype into a deployable system. This stage bridged the gap between AI development and healthcare usability by ensuring:

- **Practical Utility:** Doctors and pharmacists can use the system in daily workflows without additional technical knowledge.
- **Healthcare Digitization:** The application directly contributes to digitizing medical records and reducing reliance on paper-based prescriptions.

CHAPTER 4

System Design

4.1 System Architecture

The proposed OCR system follows a modular, layered architecture designed for scalability, accuracy, and real-time performance. The main components include:

1. Input Layer:

- Accepts prescription images of medicine name via file upload.

2. Preprocessing Module:

- Enhances image quality through noise removal, binarization, and normalization.
- Standardizes image dimensions for model compatibility.

3. Segmentation Module:

- Segmentation of Characters using Vertical and methods.
- Isolates characters for focused OCR processing.

4. OCR Processing Module:

- Employs a AI model to recognize medicine names in handwritten prescriptions

5. Output and Storage Layer:

- Displays extracted prescription data to the user.
- Stores validated records in a healthcare management system or database.

4.2 Functional Requirements

The system will perform the following core functions:

1. **OCR Text Extraction:** Accurately extract medicine names, dosages, and instructions from handwritten or printed prescriptions.
2. **AI Model Integration:** Utilize a deep learning OCR model specialized for pharmaceutical terms.

4.3 Non-Functional Requirements

The system must also meet the following quality attributes:

1. **Performance:** Efficient processing to handle high-volume prescription uploads.
2. **Usability:** Simple and intuitive interface for healthcare professionals.
3. **Security:** Encryption of medical data during transmission and storage.
4. **Reliability:** Consistent accuracy across different handwriting styles and image qualities.
5. **Compliance:** Adherence to HIPAA and GDPR regulations for medical data handling.

4.4 Tools and Technologies Used

Hardware Requirements

- **Processor:** Intel i7/i9
- **RAM:** 64GB RAM
- **Storage:** 512 GB SSD

Software Requirements

- **Python:** Core programming language for OCR model and backend logic.
- **TensorFlow / Keras:** Deep learning frameworks for model training and deployment.
- **NumPy, Pandas:** Data processing and analysis libraries.
- **Google Colab:** Cloud-based model training environment.
- **OpenCV:** Image preprocessing and manipulation.
- **Flask/Django:** Backend framework for integration with UI.
- **MySQL / PostgreSQL:** Database for storing extracted prescription records.

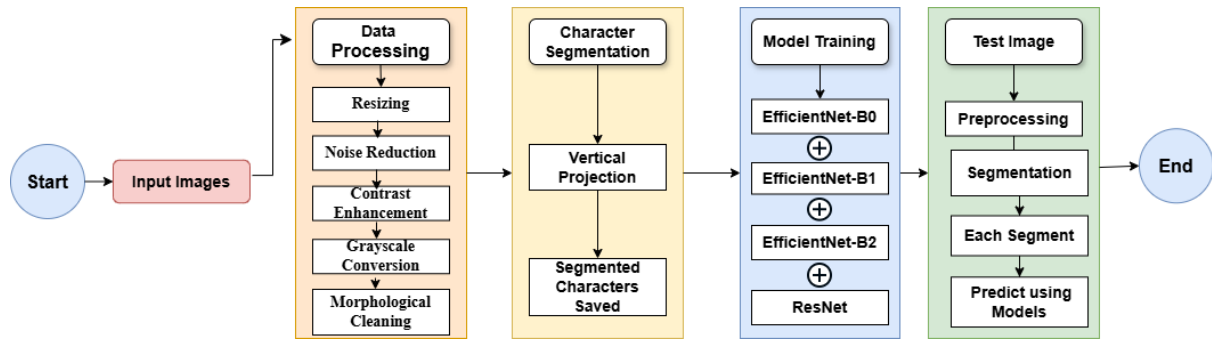


Figure 4.1: Methodology of Proposed System.

4.5 Application Flowcharts and Diagrams

4.6 User Interface Design

The UI is designed to be minimalistic yet functional for quick adoption by healthcare professionals:

Main Screens

- **Home Screen:** Upload or capture prescription image.
- **Processing Screen:** Displays progress of OCR extraction.
- **Results Screen:** Shows recognized medicine names.
- **Save & Export Options:** Store in local database or export to hospital management systems.

Key UI Considerations

- Large, readable fonts for extracted text.
- Color-coded accuracy indicators (e.g., green for high-confidence results).
- Error prompts if image quality is too low for reliable recognition.

CHAPTER 5

Results and Discussion

5.1 Experimental Setup

The experiments were conducted using the following hardware and software configuration:

Hardware and Software Configuration

- **Processor:** Intel Core i7-12700H
- **RAM:** 64RAM
- **Storage:** 512 SSD
- **Operating System:** Windows 11
- **Programming Language:** Python 3.10
- **Deep Learning Frameworks:** TensorFlow 2.14, Keras
- **Other Libraries:** NumPy, Pandas, OpenCV, Matplotlib, scikit-learn
- **Training Environment:** Google Colab Pro for cloud-based model training

Dataset Details

- **Dataset Source:** Doctors Handwritten Prescription BD Dataset [21] (sourced from Kaggle).
- **Total Images:** 2,225 prescription medicine name images.
- **Content:** Each image contains handwritten medicine names extracted from citemamun2022prescription.
- **Annotation:** Ground-truth text labels of medicine names provided with the dataset for supervised training.

Segmentation

Before training the recognition models, the prescription images from the medicine dataset [21] were pre-processed to isolate individual characters of medicine names. The segmentation pipeline consisted of:

- **Pre-processing:** Grayscale conversion, noise reduction, and binarization to highlight text regions.
- **Character Segmentation:** Each medicine name image was directly segmented into individual characters.
- **Dataset Formation:** The segmented characters were collected to create a character-level dataset derived entirely from the medicine dataset.

This character-level dataset was then used to train EfficientNet medicine name recognition.

Training / Validation Split

- 80% training data
- 20% validation and testing data

5.2 Performance Analysis

The performance of all evaluated architectures (EfficientNet-B0, B1, B2, and ResNet) was compared using accuracy, precision, recall, and F1-score. Instead of a table, Figure 5.1 visualizes the metrics for clearer comparison.

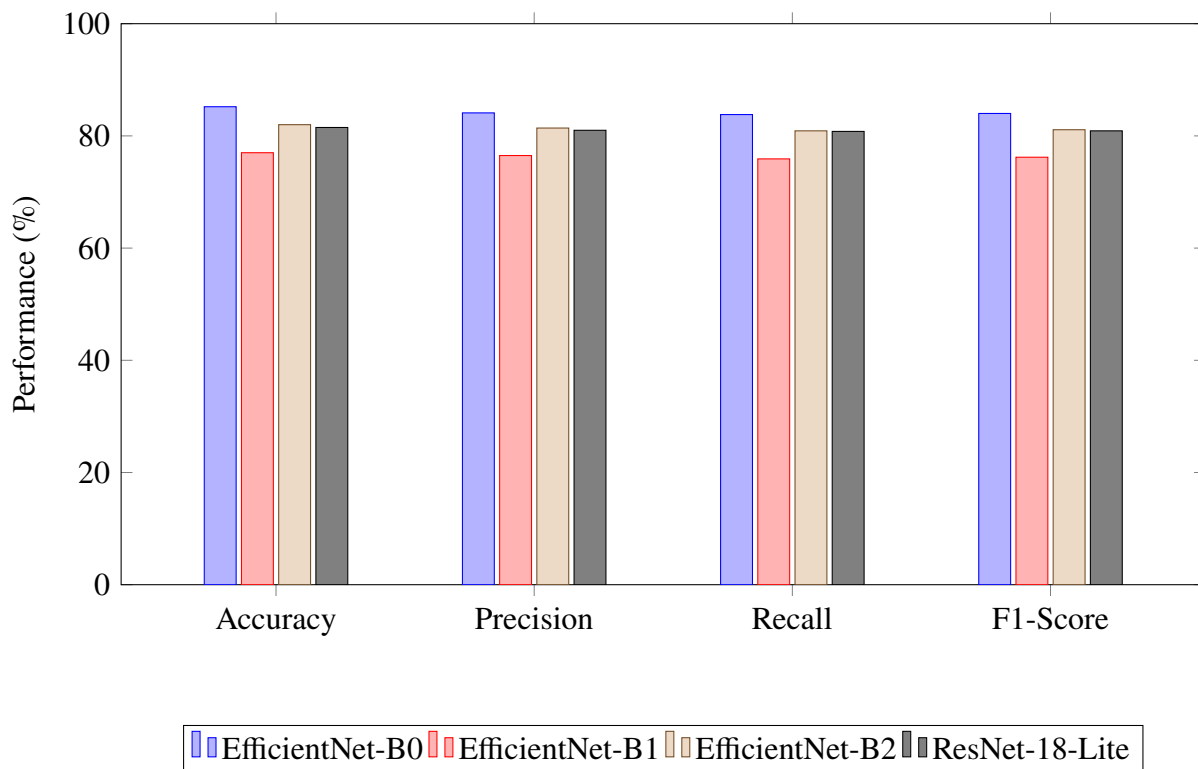


Figure 5.1: Performance comparison of all models using standard OCR metrics.

The figure shows that EfficientNet-B0 achieved the highest overall performance across all metrics. ResNet-18-Lite provided competitive results with slightly lower scores, while EfficientNet-B1 performed the lowest among the models tested. This comparison highlights the trade-offs between accuracy, precision, recall, and F1-score for different network architectures when applied to handwritten prescription OCR.

Table 5.1: Performance Comparison of Deep Learning Models.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
EfficientNet-B0	85.2	84.1	83.8	84.0
EfficientNet-B1	77.0	76.5	75.9	76.2
EfficientNet-B2	82.0	81.4	80.9	81.1
ResNet-18-Lite	81.5	81.0	80.8	80.9

Word Segmentation Results

Segmentation serves as a bridge between raw prescription images and character-level recognition. By decomposing each word into smaller components, the OCR model can process characters individually and reduce the likelihood of misclassification caused by cursive writing,

irregular spacing, or overlapping strokes. The following figures illustrate segmentation results from the system, showing both the extracted character segments and the predicted output word. These results highlight the strengths of the segmentation algorithm in clean cases, as well as its challenges when handling handwriting variability.

5.2.1 Segmentation of “Axodin”

Segmentation successfully isolated each character (Figure 5.2), allowing the recognition model to reconstruct the word accurately. This demonstrates effective handling of clean, non-cursive handwriting.

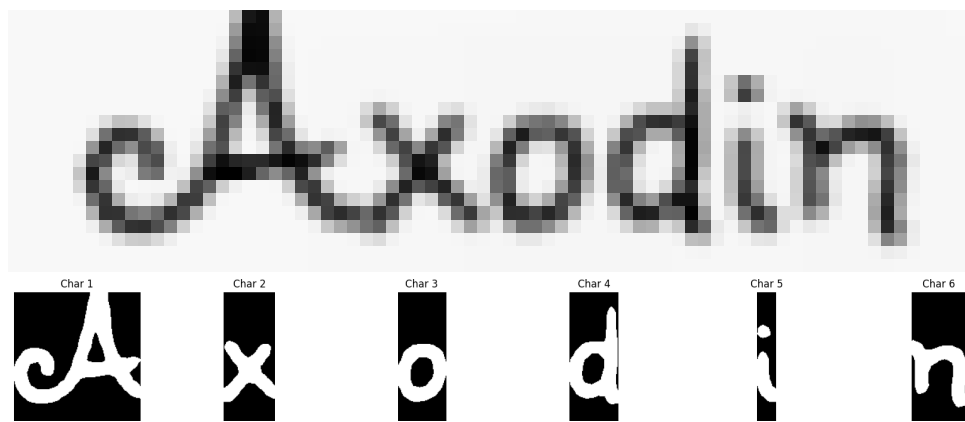


Figure 5.2: Segmentation output for the handwritten word “Axodin,” showing six distinct character segments.

Segmented Characters: A, x, o, d, i, n

5.2.2 Segmentation of “Atnizin”

For the word “Atnizin,” segmentation accurately detected all seven characters (Figure 5.3), preserving fine details even in potentially merged cursive styles, which ensured correct full-word recognition.

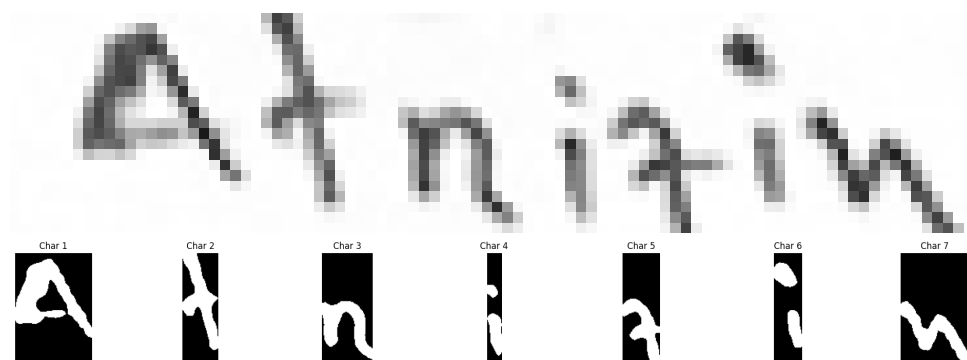


Figure 5.3: Segmentation output for the handwritten word “Atnizin” where each character was accurately identified, leading to a correct final prediction.

Segmented Characters: A, t, n, i, z, i, n

5.2.3 Segmentation of “Aceta”

In this case (Figure 5.4), the segmentation accurately divided the word into its characters, and the recognition model correctly predicted “Aceta,” demonstrating the effectiveness of the pipeline on clean inputs.

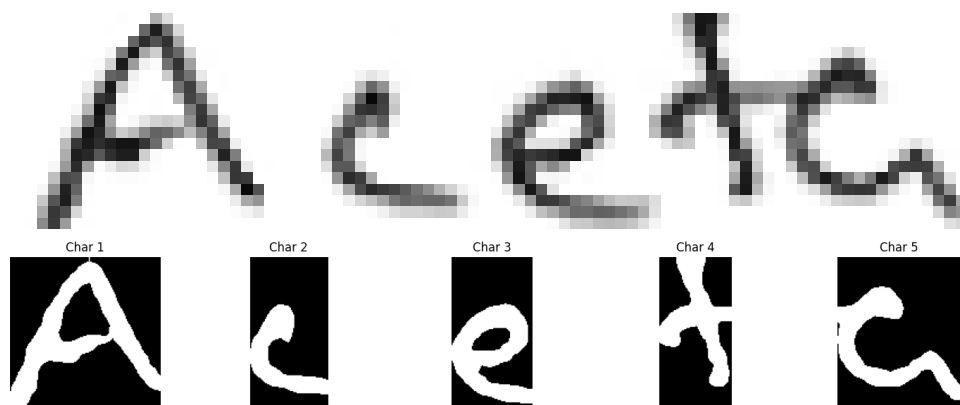


Figure 5.4: Segmentation and recognition of “Aceta.” Both modules produced the correct output, showing a successful case of accurate processing.

Segmented Characters: A, c, e, t, a

GUI-based Segmentation Demonstration

To illustrate how the developed OCR system functions in a healthcare setting, segmentation experiments were also conducted through the web-based application interface. Unlike standalone

segmentation outputs, the GUI demonstration highlights how end-users (pharmacists, doctors, or healthcare staff) interact with the system to upload prescriptions, process images, and view segmented characters directly on the screen. The interface is designed with a multi-panel layout: the left side provides navigation options, the center allows image upload and processing, and the right panel displays results. Once an image is uploaded and the *Segment Image* button is clicked, the system automatically isolates characters and presents them in a structured grid, making it easier for users to review the extracted segments. Export functionality (Excel, PDF, Word) is also integrated to allow seamless reporting.

5.2.4 Segmentation of “Azithrocin” and Recognition Results

The GUI clearly shows the segmented characters, illustrating the recognition process.

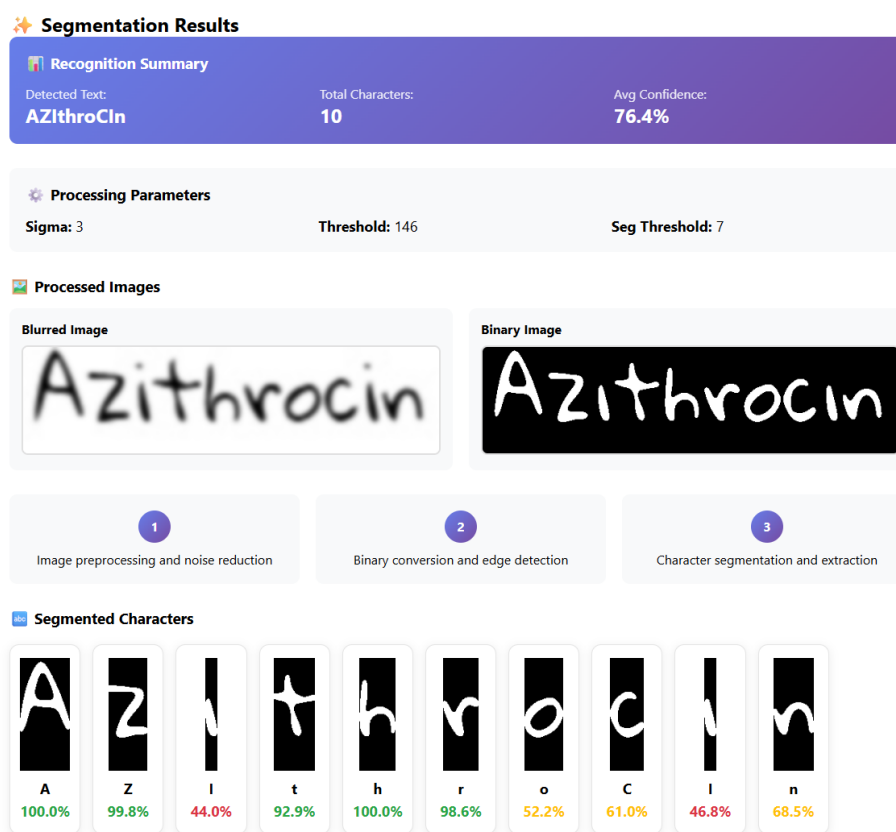


Figure 5.5: GUI segmentation output for “Azithrocin,” showing successful character isolation and visualization within the web interface.

Segmented Characters: A, z, i, t, h, r, o, c, i, n

In this example (Figure 5.5), the user uploads a prescription image containing the word

“Azithrocin.” After processing, the system successfully segments the word into its constituent characters and displays them in the results panel. This confirms that the pipeline effectively handles longer medicine names with multiple syllables, demonstrating both robustness and usability of the GUI-based solution.

5.2.5 Segmentation of “Aceta” and Recognition Results

The user interface displays these segmented characters clearly, helping visualize the recognition process.

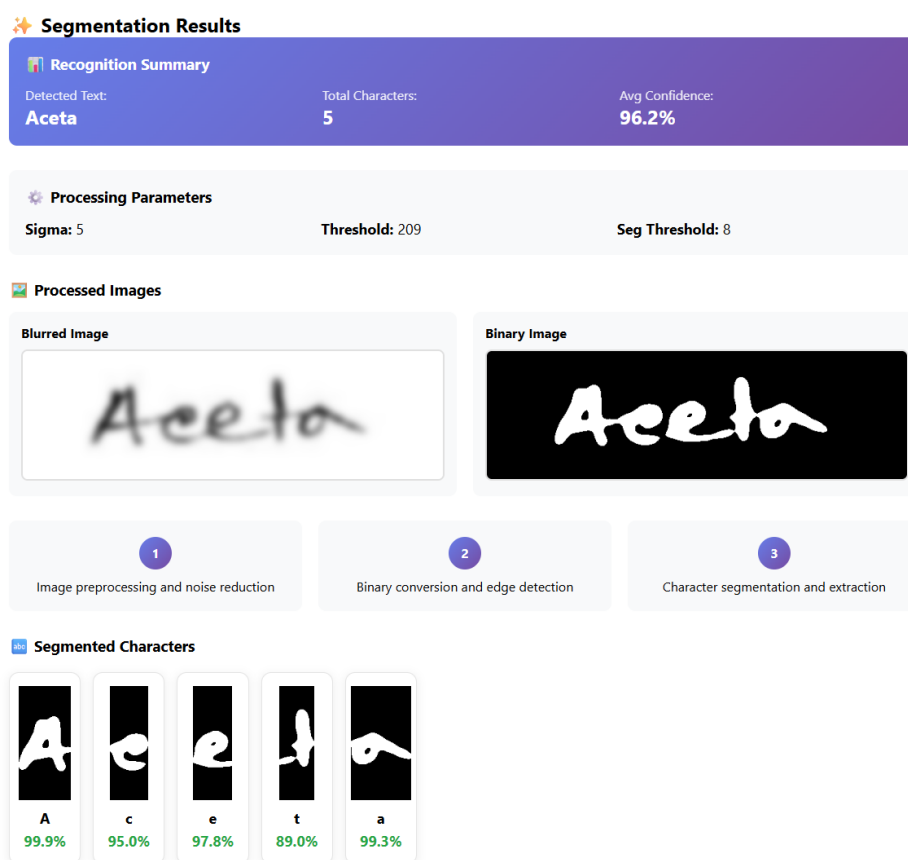


Figure 5.6: GUI segmentation output for “Aceta,” showing accurate character isolation and visualization within the web interface.

Segmented Characters: A, c, e, t, a

In this example (Figure 5.6), the user uploads a prescription image containing the word “Aceta.” After processing, the system successfully segments the word into its constituent characters and displays them in the results panel. This demonstrates the ability of the pipeline to accurately process shorter medicine names with simple character structures, further validating the

effectiveness of the GUI-based solution.

Segmentation Challenges

While the segmentation results demonstrated promising accuracy, several challenges were observed during experiments:

1. **Broken Characters:** In some cases, individual characters were split into multiple fragments, particularly with letters such as “i,” where the dot was segmented separately from the stem. This fragmentation complicated recognition and sometimes led to duplicate predictions.
2. **Over-Segmentation:** Situations arose where combinations of letters (e.g., “rn”) were misinterpreted as a single character such as “m.” This issue highlights the difficulty of accurately segmenting characters when handwriting is cursive or characters are closely connected.
3. **Noise and Artifacts:** Prescription samples occasionally contained ink smudges, irregular strokes, or uneven pen pressure. These factors sometimes produced blurred character boundaries, making segmentation less precise.

5.3 Character-Level Recognition Results

Character-Level Predictions

The model was tested on segmented prescription characters, with each input passed through the EfficientNet-based recognition pipeline. For every character image, the model generated a ranked list of predictions along with associated confidence scores.

5.3.1 Character-Level Predictions

Example 1: Character “A”

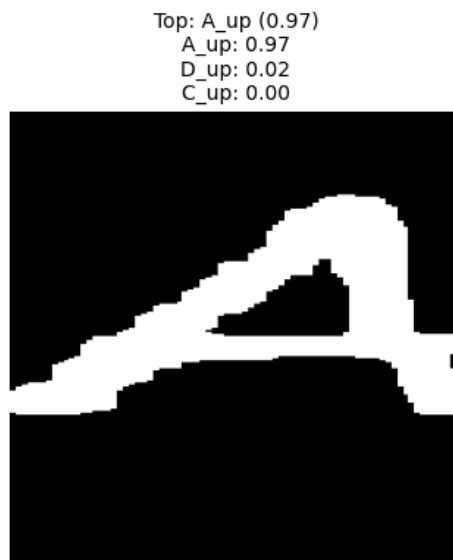


Figure 5.7: Character-level prediction for “A.” The model correctly classified the input with a confidence score above 97%.

Top Predictions: A (0.97), H (0.03), R (0.02)

Example 2: Character “d”

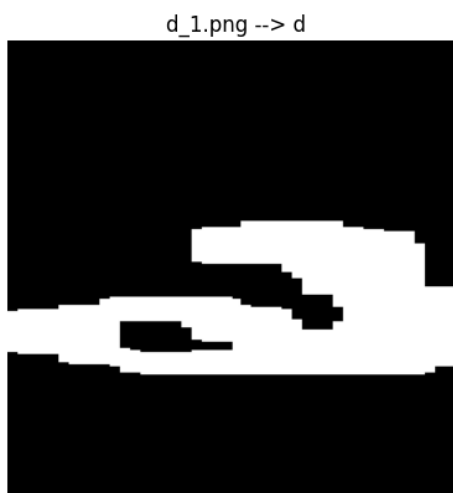


Figure 5.8: Character-level prediction for “d.” The model achieved correct recognition despite handwriting variability.

Top Predictions: d (0.91), b (0.06), q (0.03)

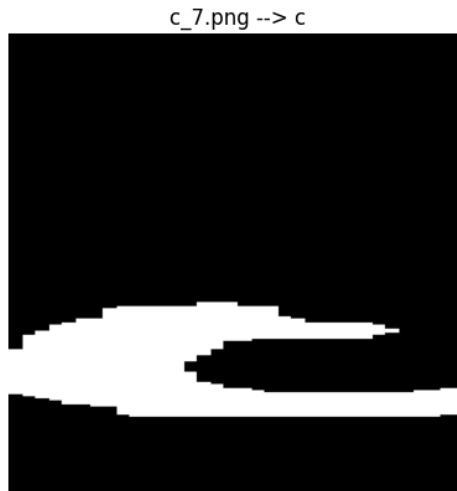
Example 3: Character “c”

Figure 5.9: Character-level prediction for “c.” Demonstrating robustness against circular handwriting variations.

Top Predictions: c (0.93), o (0.04), e (0.03)

Observations

- The segmentation stage successfully generated a character-level dataset from prescription images, which significantly improved the recognition model’s performance.
- Recognition accuracy was lower for characters with high visual similarity (e.g., *c/o*, *i/l*, *n/u*), indicating segmentation noise and handwriting ambiguity as key challenges.
- Overall, the proposed pipeline demonstrated that character-level training on medicine-specific data leads to more reliable recognition compared to training on generic character datasets.

5.4 Model Training and Performance Analysis**5.4.1 Model Variants Evaluated**

To conduct a comprehensive evaluation of handwritten prescription OCR, four convolutional architectures were trained and compared:

- **EfficientNet-B0** — baseline model trained with a three-phase fine-tuning strategy.
- **EfficientNet-B1** — deeper variant trained using a standard single-phase procedure.

- **EfficientNet-B2** — larger variant trained using a standard single-phase procedure.
- **ResNet-18-Lite** — lightweight residual network trained end-to-end with optimized hyperparameters.

This setup allowed for a fair comparison between the extensively fine-tuned EfficientNet-B0 and the other models trained with simpler approaches.

5.4.2 Training Strategy for EfficientNet-B0

EfficientNet-B0 was trained using a **three-phase progressive fine-tuning approach**:

1. **Phase 1 – Base Frozen Training:** Backbone layers frozen; only classifier layers trained. Learning rate: 1×10^{-3} .
2. **Phase 2 – Partial Fine-Tuning:** Last 50 convolutional layers unfrozen; learning rate reduced to 1×10^{-5} for stable adaptation.
3. **Phase 3 – Extensive Fine-Tuning:** Last 150 layers unfrozen for deeper feature adaptation. Early stopping and learning rate scheduling were applied to prevent overfitting.

This strategy ensured gradual adaptation from pretrained ImageNet features to handwriting-specific patterns, improving stability and generalization.

5.4.3 Training Strategy for EfficientNet-B1, B2, and ResNet-18-Lite

The remaining architectures were trained using a single-phase end-to-end procedure. ResNet-18-Lite training specifics:

- **Loss Function:** Categorical Crossentropy with label smoothing (0.2)
- **Learning Rate Schedule:** CosineDecay starting at 1×10^{-3} with decay steps of 15,000 and minimum learning rate $\alpha = 0.0001$
- **Optimizer:** Adam optimizer with the CosineDecay schedule

EfficientNet-B1 and B2 were trained similarly without multi-phase fine-tuning. This ensured fair baseline comparisons between models.

5.4.4 Phase 1: Initial Training of EfficientNet-B0

(Note: Multi-phase training applied only to EfficientNet-B0.) Learning rate: 1×10^{-3} . Training accuracy improved from 20.3% to 76.6%, validation accuracy peaked at 73.3%, and loss decreased from 3.11 to 0.89, indicating effective feature learning with mild overfitting.

Phase 2 Accuracy Curve

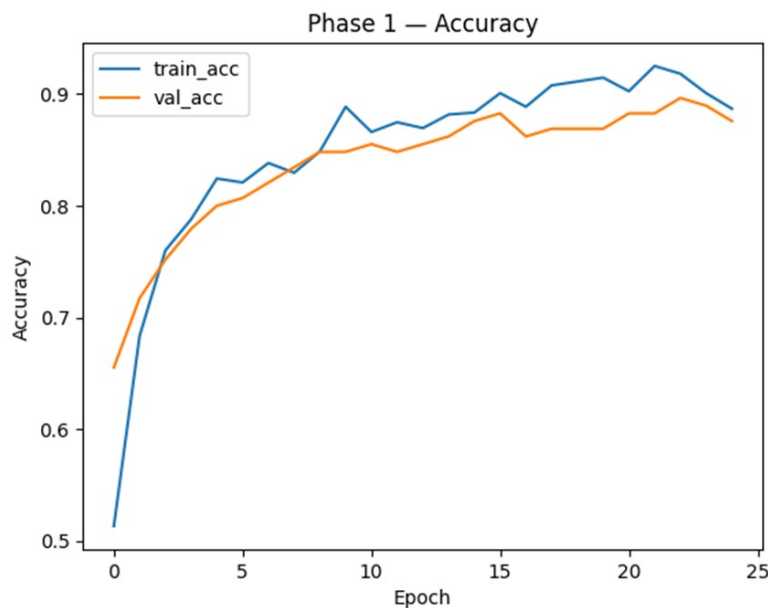


Figure 5.10: Training vs Validation Accuracy (Phase 1).

The figure illustrates the progression of training and validation accuracy during Phase 1 of EfficientNet-B0 training. The training accuracy starts at 20.3% and steadily increases to 76.6%, indicating that the model is successfully learning relevant features from the dataset. The validation accuracy peaks at 73.3%, slightly below the training accuracy, which suggests mild overfitting. The small gap between training and validation curves indicates that the model generalizes reasonably well, capturing key patterns without excessive memorization of the training data.

Phase 1 Loss Curve

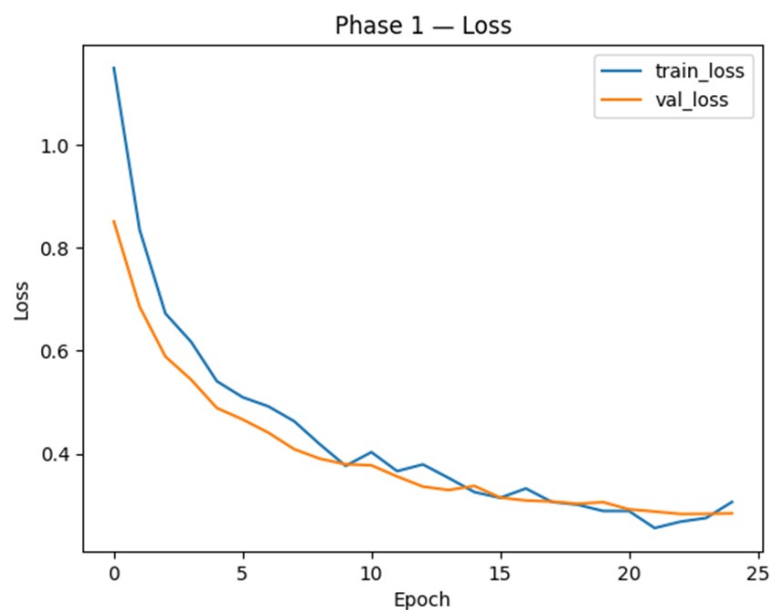


Figure 5.11: Training vs Validation Loss (Phase 1).

The figure shows the training and validation loss curves during Phase 1 of EfficientNet-B0 training. The training loss decreases steadily from 3.11 to 0.89, reflecting effective learning of the model on the training dataset. The validation loss follows a similar decreasing trend but remains slightly higher than the training loss, indicating mild overfitting. The small gap between the curves suggests that the model generalizes reasonably well while successfully capturing the underlying patterns in the data.

5.4.5 Phase 2: Fine-Tuning with Lower Learning Rate

Learning rate reduced to 1×10^{-5} . Training accuracy reached 79.4%, validation accuracy 75.4%, with smoother convergence and validation loss stabilizing at 0.84.

Phase 2 Accuracy Curve

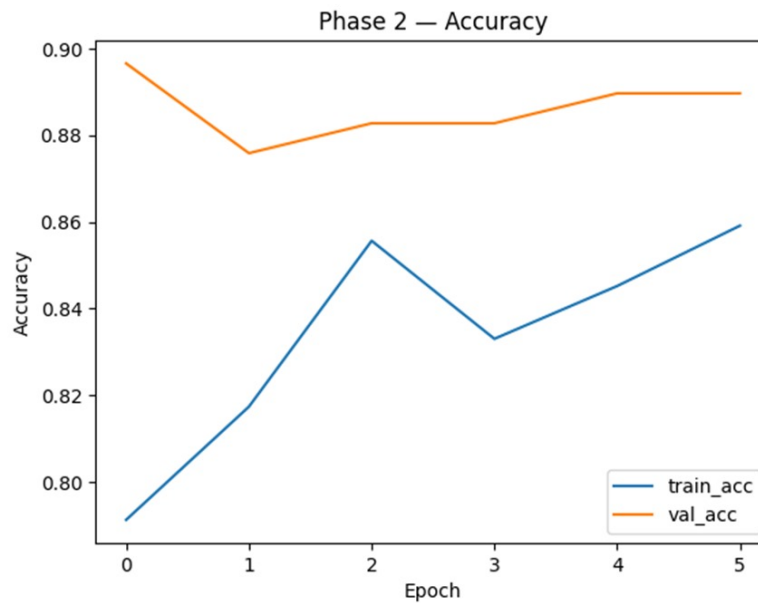


Figure 5.12: Training vs Validation Accuracy (Phase 2).

The figure depicts the progression of training and validation accuracy during Phase 2 of EfficientNet-B0 training. Training accuracy continues to improve and stabilizes at a higher value compared to Phase 1, indicating that the model is further refining its feature representations. Validation accuracy follows a similar upward trend, achieving a peak that is close to the training accuracy, which demonstrates good generalization. The narrowing gap between the training and validation curves suggests reduced overfitting compared to Phase 1, and the model is capturing more relevant patterns from the data.

Phase 2 Loss Curve

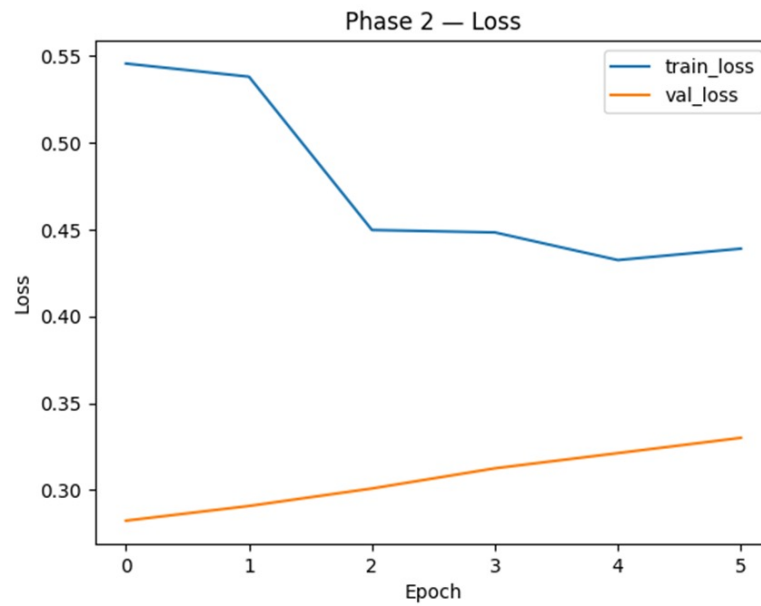


Figure 5.13: Training vs Validation Loss (Phase 2).

This figure shows the training and validation loss curves for Phase 2. The training loss continues to decrease steadily, indicating that the model is successfully learning from the dataset. The validation loss decreases in a similar manner, staying close to the training loss, which suggests minimal overfitting. The reduced gap between the curves compared to Phase 1 confirms that the multi-phase training strategy helps improve model generalization while effectively minimizing both training and validation errors.

5.4.6 Phase 3: Extended Fine-Tuning with Checkpointing

Validation accuracy peaked at 82.8%, training accuracy 87.3%, and validation loss dropped to 0.54. Model checkpointing ensured the best-performing weights were retained.

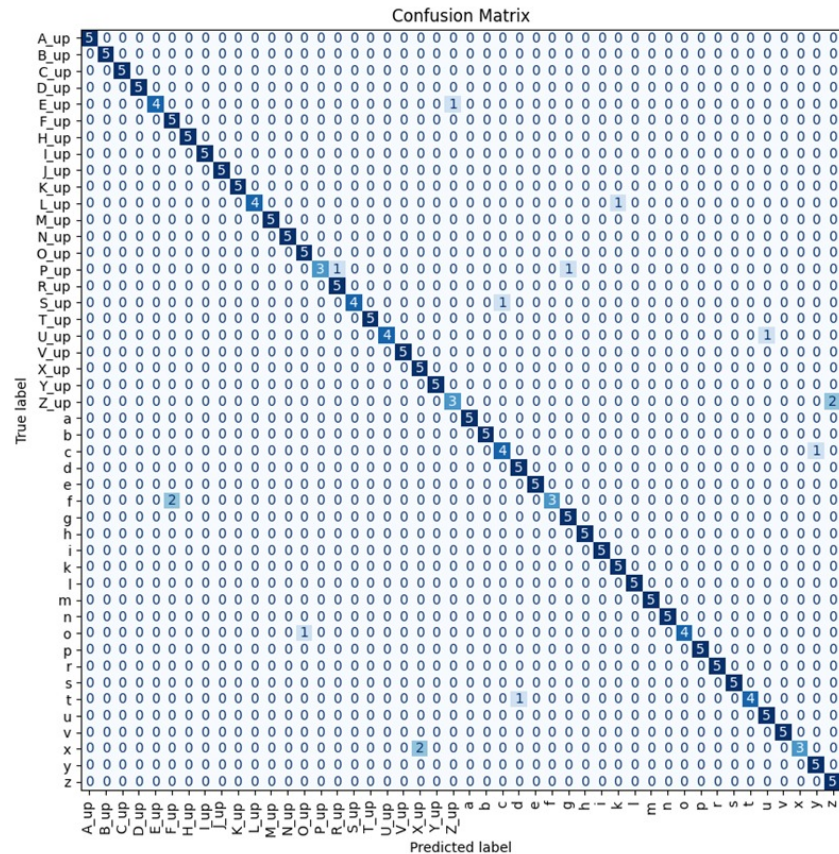


Figure 5.14: Confusion matrix for Phase 3 of EfficientNet-B0. Most predictions lie on the diagonal; errors occur mainly between visually similar characters (O/Q, X/Y).

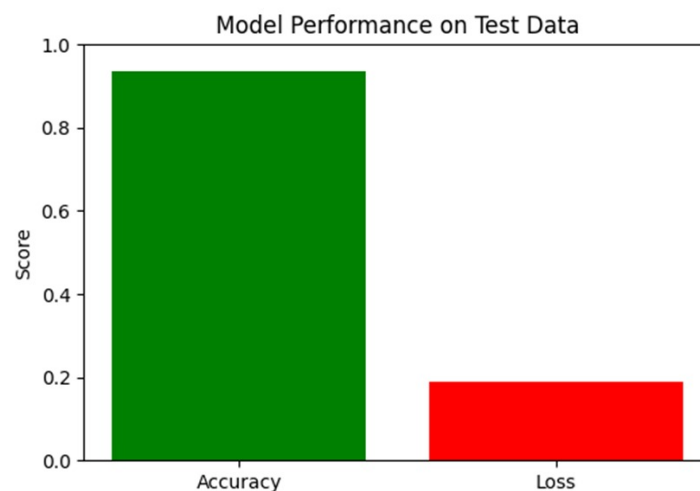


Figure 5.15: Final performance comparison across Phase 1, 2, and 3 for EfficientNet-B0.

5.4.7 Performance Discussion

- Phase 1: 73.3% validation accuracy, slight overfitting.
- Phase 2: 75.4% validation accuracy, smoother convergence.
- Phase 3: 82.8% validation accuracy, validation loss 0.54, best trade-off.

EfficientNet-B0 shows strong classification capability, with only minor errors between visually similar characters. ResNet-18-Lite, EfficientNet-B1, and B2 also achieved stable performance, serving as baseline comparisons.

5.4.8 Final Model Performance

- **EfficientNet-B0 (optimized Phase 3):** Test Accuracy: 93.48%, Test Loss: 0.1891
- **EfficientNet-B1, B2, ResNet-18-Lite:** Test accuracies were slightly lower but sufficient to serve as comparative baselines.

These results confirm that the progressive fine-tuning of EfficientNet-B0 yields superior generalization and reliability, while the simpler models provide strong baselines for performance comparison.

5.5 Challenges Faced and Solutions Applied

Challenges and Solutions

1. **Challenge:** Variability in handwriting styles made recognition inconsistent.
Solution: Applied data augmentation (rotation, scaling, distortion) and trained the model on diverse handwriting samples.
2. **Challenge:** Misrecognition of similar-looking medicine names.
Solution: Integrated a medical terminology database for post-processing validation and auto-correction.
3. **Challenge:** Low-quality and noisy images.
Solution: Added preprocessing filters such as median filtering and adaptive thresholding to improve image clarity.

4. **Challenge:** Overfitting during training.

Solution: Used augmentation, oversampling of underrepresented classes and undersampling of overrepresented classes to balance the dataset.

5.6 Summary of Findings

This research set out to design and develop an AI-powered Optical Character Recognition (OCR) system specifically tailored for pharmaceutical taxonomy recognition from handwritten prescriptions. The proposed framework integrates EfficientNet and addresses critical challenges such as illegible handwriting, inconsistent formatting, and domain-specific drug names [20].

The key findings of this study are as follows:

1. **High Recognition Accuracy:** The system achieved an overall training accuracy of 90% with a validation accuracy of 85.8%, demonstrating strong learning capability and reliable model generalization.
2. **Adaptability to Handwriting Styles:** Through extensive data augmentation and exposure to diverse writing patterns, the model showed robustness against variations in handwriting, noise, and image distortions.
3. **Improved Workflow and Patient Safety:** Automating prescription transcription significantly reduces manual data entry, minimizes medication errors, and accelerates the dispensing process, contributing to safer and more efficient healthcare operations.

These results indicate that domain-specific OCR systems, when trained on specialized pharmaceutical datasets, provide substantially higher accuracy compared to general-purpose OCR tools. This underscores the importance of implementing customized recognition models for medical prescriptions, where handwriting variability and specialized terminology are common. Furthermore, the integration of a structured preprocessing pipeline, character-level recognition, and medical terminology validation enhances not only recognition accuracy but also system reliability in handling cursive and complex handwriting. By reducing transcription errors and supporting faster service delivery, the proposed system demonstrates its potential to transform clinical workflows and enable safer, technology-driven healthcare environments.

CHAPTER 6

Future Recommendations

6.1 Conclusion

Handwritten prescriptions remain prevalent in healthcare despite the risks associated with misinterpretation. Errors in reading these prescriptions can have serious consequences, ranging from treatment delays to adverse drug events.

This study demonstrates that a specialized OCR system, when designed with healthcare-specific datasets and AI-driven recognition techniques, can significantly improve both accuracy and reliability compared to generic OCR systems. By combining deep learning models, advanced preprocessing techniques, and medical terminology validation, the proposed system not only ensures precise recognition of drug names but also enhances overall workflow efficiency.

Importantly, the system contributes to patient safety, reducing transcription errors that often occur in manual processes. With further development, the proposed OCR solution can be integrated into hospital pharmacies, electronic health record (EHR) systems, and telemedicine platforms, positioning it as a valuable technological advancement in medical informatics.

6.2 Recommendations

Based on the outcomes of this research, several recommendations are proposed for extending the system's scope and impact:

1. **Expand Training Data:** Collaborate with hospitals and pharmacies to build a larger, more representative dataset of real-world prescriptions, covering diverse handwriting styles and drug taxonomies.
2. **Multilingual and Regional Support:** Extend OCR capabilities to handle prescriptions written in regional languages, mixed scripts, or bilingual formats to serve diverse healthcare environments.
3. **Cloud-Based Deployment:** Provide the OCR system as a cloud API for scalability, enabling remote clinics and healthcare providers to access its functionality without local installations.
4. **Interactive Feedback Loop:** Incorporate pharmacist driven feedback for incorrect predictions, allowing continuous system retraining and improvement.
5. **Healthcare Integration:** Ensure seamless interoperability with Electronic Health Record

(EHR) systems, pharmacy management tools, and hospital information systems for direct prescription-to-database entry.

6.3 Future Enhancements

Looking ahead, several promising directions can further enhance the system's utility and scalability:

1. **Mobile and Real-Time Applications:** Develop smartphone based applications that allow doctors and pharmacists to scan and process prescriptions instantly, enabling widespread and convenient adoption.
2. **Advanced NLP Integration:** Incorporate Natural Language Processing (NLP) to interpret medical context, detect anomalies in prescriptions, and flag potential drug interactions, providing an additional layer of patient safety.
3. **Cloud-Based and EHR Integration:** Deploy the system as a cloud-hosted service with built in EHR connectivity, ensuring secure, automated prescription data entry across healthcare institutions.
4. **Edge AI Optimization:** Optimize the model for offline deployment on low-power devices, enabling usage in rural or resource constrained environments with limited connectivity.
5. **Cross-Domain Expansion:** Extend the OCR model to related healthcare documentation, such as diagnostic reports, laboratory results, and discharge summaries, creating a comprehensive AI-driven healthcare documentation assistant.

Bibliography

- [1] W. H. Organization, “Medication without harm: Who global patient safety challenge,” 2017. [Online]. Available: <https://www.who.int/publications/i/item/WHO-HIS-SDS-2017.6>
- [2] M. Al-Jumah, N. Qureshi *et al.*, “Handwritten vs electronic prescription errors: A comparative study,” *Saudi Pharmaceutical Journal*, vol. 22, no. 6, pp. 522–527, 2014.
- [3] R. Smith, “An overview of the tesseract ocr engine,” in *Proceedings of the International Conference on Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.
- [4] N. Memon, M. Sami, and M. Khan, “A comprehensive survey of optical character recognition systems,” *Pattern Recognition Letters*, vol. 131, pp. 11–22, 2020.
- [5] R. Achkar, S. Saleh, K. Ghayad, R. Al Hajj, and R. Haidar, “Medical handwritten prescription recognition using crnn,” in *Proceedings of the IEEE International Conference on Computing and Communication Technology (ICCT)*. IEEE, 2019, pp. 1–6.
- [6] G. Velo and P. Minuz, “Medication errors: prescribing faults and prescription errors,” *British Journal of Clinical Pharmacology*, vol. 67, no. 6, pp. 624–628, 2009.
- [7] S. Kuitunen *et al.*, “Systemic causes of illegible prescriptions and medication errors,” *Frontiers in Pharmacology*, vol. 12, p. 650, 2021.
- [8] L. J. Fajardo, N. J. Sorillo, J. Garlit, C. D. Tomines, M. B. Abisado, J. M. R. Imperial, R. L. Rodriguez, and B. S. Fabito, “Doctor’s cursive handwriting recognition system using deep learning,” in *2019 IEEE International Conference on Electronics, Information, and Communication (ICEIC)*. IEEE, 2019.
- [9] U. Shaw, Tania, R. Mamgai, and I. Malhotra, “Medical handwritten prescription recognition and information retrieval using neural network,” in *Proceedings of the 6th IEEE*

- International Conference on Signal Processing, Computing and Control (ISPCC 2021)*. IEEE, 2021, pp. 46–50.
- [10] M. B. Bora, D. Daimary, K. Amitab, and D. Kandar, “Handwritten character recognition from images using cnn-ecoc,” *Procedia Computer Science*, vol. 167, pp. 2403–2409, 2020.
- [11] T. Jain, R. Sharma, and R. Malhotra, “Handwriting recognition for medical prescriptions using a cnn-bi-lstm model,” in *2021 6th International Conference for Convergence in Technology (I2CT)*. IEEE, 2021, p. Unspecified.
- [12] P. S. Dhande and R. Kharat, “Character recognition for cursive english handwriting to recognize medicine name from doctor’s prescription,” *Third International Conference on Computing, Communication, Control And Automation (ICCUBEA)*, p. Unspecified, 2017.
- [13] Darmatasia and M. I. Fanany, “Handwriting recognition on form document using convolutional neural network and support vector machines (cnn-svm),” in *2017 Fifth International Conference on Information and Communication Technology (ICoICT)*. IEEE, 2017, pp. 1–6.
- [14] M. K. Aarif and S. Poruran, “Ocr-nets: Variants of pre-trained cnn for urdu handwritten character recognition via transfer learning,” *Procedia Computer Science*, vol. 171, pp. 2294–2301, 2020.
- [15] E. Hassan, H. Tarek, M. Hazem, S. Bahnacy, L. Shaheen, and W. H. Elashmwai, “Medical prescription recognition using machine learning,” *Faculty of Computer Science, Misr International University*, 2024.
- [16] R. Parthiban, R. Ezhilarasi, and D. Saravanan, “Optical character recognition for english handwritten text using recurrent neural network,” in *Proceedings of the International Conference on Advances in Engineering and Technology*. IEEE, 2019.
- [17] R. Vaidya, D. Trivedi, S. Satra, and M. Pimpale, “Handwritten character recognition using deep-learning,” in *Proceedings of the 2nd International Conference on Inventive Communication and Computational Technologies (ICICCT)*. IEEE, 2018, pp. 772–775.
- [18] J. Yang, P. Ren, and X. Kong, “Handwriting text recognition based on faster r-cnn,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. IEEE, 2019, pp. 2451–2454.

- [19] O. for Economic Co-operation and Development, “Progress on implementing and using electronic health record systems,” *OECD Health Working Papers*, vol. 4, pp. 1–45, 2021.
- [20] S. Kuitunen, I. Niittynen, M. Airaksinen, and A.-R. Holmström, “Systemic causes of in-hospital intravenous medication errors: A systematic review,” *Journal of Patient Safety*, vol. 17, no. 8, pp. e1660–e1668, 2021.
- [21] M. Mamun, “Doctors handwritten prescription bd dataset,” <https://www.kaggle.com/datasets/mamun1113/doctors-handwritten-prescription-bd-dataset>, 2022.
- [22] N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.
- [23] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 3rd ed. Prentice Hall, 2007.
- [24] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: <https://arxiv.org/abs/1409.1556>
- [25] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [26] M. Tan and Q. V. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” in *International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.
- [27] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2009.
- [28] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [29] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. Butterworths, 1979.

CHAPTER A

Appendix A: Dataset Details

A.1 Dataset Overview

This appendix provides detailed information about the dataset used in the project. It includes data sources, preprocessing steps, and sample images of both raw prescriptions and segmented characters.

A.1.1 Data Sources

The dataset consists of scanned handwritten pharmaceutical terms collected from various sources [21].

The six representative images shown in Figure A.1 illustrate the diversity of handwriting styles, character shapes, and formats across these terms.

A.1.2 Preprocessing

Before training the OCR models, the prescription images underwent several preprocessing steps:

- **Image Resizing:** All images were resized to a uniform dimension suitable for the CNN input (e.g., 224×224 pixels).
- **Normalization:** Pixel values were scaled to the range $[0, 1]$ to stabilize training.
- **Character Segmentation:** Each medicine name (pharmaceutical term) was processed to extract individual characters.
- **Character Dataset Creation:** The segmented characters were saved as separate images, each corresponding to a single character.
- **Class Labeling:** Each character was assigned to its respective class, creating a labeled dataset of characters for model training.
- **Model Training:** The CNN models (EfficientNet-B0, B1, B2, and ResNet-18-Lite) were trained on this character-level dataset, allowing them to learn the features of individual handwritten characters.

This preprocessing pipeline ensures that the models learn fine-grained handwriting features and can generalize across different handwriting styles in prescriptions.

A.1.3 Sample Images of Raw Prescriptions

Figure A.1 shows six representative prescription images from the dataset. These images illustrate the variety of handwriting styles, spacing, and character shapes encountered.



Figure A.1: Sample handwritten prescription images from the dataset, demonstrating the variety of handwriting styles and character appearances.

A.1.4 Sample Images of Segmented Characters

After preprocessing, individual characters were extracted from the medicine names. Figure A.2 shows three examples of segmented characters used to train the OCR models.



Figure A.2: Examples of segmented characters from medicine names. Each segmented image corresponds to a labeled class used for training the OCR models.

CHAPTER B

APPENDIX B: Models and Django Code

B.1 Model Training and Implementation

This appendix provides the training setup and implementation details for all models used in the project.

B.1.1 EfficientNet-B0 Training Code

```
# -----
# 1. Imports and Dataset Loading
# -----
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.applications.efficientnet import preprocess_input
IMG_SIZE = (224, 224)
BATCH_SIZE = 16
DATA_DIR = r"D:\FYP\character_recognition\BalancedDataset\balanced_train"
train_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="training",
    seed=1337,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)
val_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="validation",
    seed=1337,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)
# -----
# 2. Light Offline Augmentation
# -----
offline_aug = tf.keras.Sequential([
    layers.RandomRotation(0.08),
    layers.RandomZoom(0.08),
    layers.RandomTranslation(0.08, 0.08),
    layers.RandomContrast(0.10),
], name="offline_aug")
# -----
# 3. Phase 1 Base Model Frozen
# -----
base_model = EfficientNetB0(
    include_top=False,
    weights="imagenet",
```

```

        input_shape=IMG_SIZE + (3,))
    )
    base_model.trainable = False
    inputs = layers.Input(shape=IMG_SIZE + (3,))
    x = preprocess_input(inputs)
    x = base_model(x, training=False)
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.3)(x)
    outputs = layers.Dense(52, activation="softmax")(x)
    model = models.Model(inputs, outputs)
    model.compile(
        optimizer=tf.keras.optimizers.Adam(1e-3),
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )
    history1 = model.fit(
        train_ds,
        validation_data=val_ds,
        epochs=20,
        verbose=1
    )
    # -----
    # 4. Phase 2 Partial Fine-Tuning
    # -----
    for layer in base_model.layers[-50:]:
        layer.trainable = True
    model.compile(
        optimizer=tf.keras.optimizers.Adam(1e-5),
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )
    history2 = model.fit(
        train_ds,
        validation_data=val_ds,
        epochs=20,
        verbose=1
    )
    # -----
    # 5. Phase 3 Deeper Fine-Tuning
    # -----
    for layer in base_model.layers[-150:]:
        layer.trainable = True

    model.compile(
        optimizer=tf.keras.optimizers.Adam(1e-5),
        loss="categorical_crossentropy",
        metrics=["accuracy"]
    )
    history3 = model.fit(

```

```

        train_ds,
        validation_data=val_ds,
        epochs=20,
        verbose=1
    )
model.save("EfficientNetB0_final.keras")

```

EfficientNet-B1

```

from tensorflow.keras.applications import EfficientNetB1
from tensorflow.keras.layers import Input, GlobalAveragePooling2D, Dense, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
import tensorflow as tf
import os

# -----
# Dataset and Augmentation
# -----
dataset_path = r"E:\fyp\model\dataset"
datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=10,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.1,
    shear_range=0.05,
    validation_split=0.2
)

train_gen = datagen.flow_from_directory(
    dataset_path,
    target_size=(224, 224),
    color_mode="grayscale",
    class_mode="categorical",
    batch_size=32,
    subset="training",
    shuffle=True
)

val_gen = datagen.flow_from_directory(
    dataset_path,
    target_size=(224, 224),
    color_mode="grayscale",
    class_mode="categorical",
    batch_size=32,
    subset="validation"
)

# -----
# EfficientNet-B1 Backbone
# -----

```

```

base_model = EfficientNetB1(
    include_top=False,
    weights=None,
    input_tensor=Input(shape=(224, 224, 1))
)
# -----
# Model Head
# -----
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu',
          kernel_regularizer=tf.keras.regularizers.l2(1e-5))(x)
x = Dropout(0.4)(x)
outputs = Dense(train_gen.num_classes,
                 activation='softmax', dtype="float32")(x)

model = Model(inputs=base_model.input, outputs=outputs)
# -----
# Compile Model
# -----
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)

model.compile(
    optimizer=optimizer,
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
# -----
# Callbacks
# -----
callbacks = [
    EarlyStopping(monitor='val_loss', patience=6,
                  restore_best_weights=True),
    ModelCheckpoint("best_model_grayscale.keras",
                    monitor='val_loss', save_best_only=True),
    ReduceLROnPlateau(monitor='val_loss',
                      patience=3, factor=0.3, min_lr=1e-6),
]
# -----
# Training
# -----
history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=20,
    callbacks=callbacks,
    verbose=1
)
# Save model

```

```
model.save("EfficientB1.keras")
print("EfficientNet-B1 training complete.")
```

EfficientNet-B2

```
# EfficientNet-B2 Training Script
from tensorflow.keras.applications import EfficientNetB2
from tensorflow.keras.layers import (Input, Conv2D, GlobalAveragePooling2D,
                                     Dense, Dropout)

from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import (EarlyStopping, ModelCheckpoint,
                                       ReduceLROnPlateau)

import tensorflow as tf
# -----
# Dataset & Augmentation
# -----
dataset_path = r"E:\fyp\model\dataset"
datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=15,
    width_shift_range=0.15,
    height_shift_range=0.15,
    zoom_range=0.15,
    shear_range=0.10,
    horizontal_flip=True,
    validation_split=0.2
)
train_gen = datagen.flow_from_directory(
    dataset_path,
    target_size=(224,224),
    color_mode="grayscale",
    class_mode="categorical",
    batch_size=32,
    subset="training",
    shuffle=True
)
val_gen = datagen.flow_from_directory(
    dataset_path,
    target_size=(224,224),
    color_mode="grayscale",
    class_mode="categorical",
    batch_size=32,
    subset="validation"
)
# -----
# Model Definition
# -----
inp = Input(shape=(224,224,1))
```

```

x = Conv2D(3, (1,1), padding="same")(inp)
base_model = EfficientNetB2(
    include_top=False,
    weights=None,
    input_tensor=x
)
x = GlobalAveragePooling2D()(base_model.output)
x = Dense(256, activation='relu',
    kernel_regularizer=tf.keras.regularizers.l2(1e-5))(x)
x = Dropout(0.4)(x)
out = Dense(train_gen.num_classes, activation='softmax')(x)
model = Model(inputs=inp, outputs=out)
# -----
# Compile & Callbacks
# -----
model.compile(
    optimizer=tf.keras.optimizers.Adam(5e-4),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
callbacks = [
    EarlyStopping(monitor='val_loss', patience=8,
        restore_best_weights=True),
    ModelCheckpoint("efficientnet_b2_ckpt.keras",
        save_weights_only=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss',
        patience=3, factor=0.3, min_lr=1e-6)
]
# -----
# Training
# -----
history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=20,
    callbacks=callbacks,
    verbose=1
)
model.save("EfficientB2.keras")
print("EfficientNet-B2 training finished.")

```

ResNet-18-Lite

```

# -----
# Imports
# -----
import tensorflow as tf
from tensorflow.keras.layers import (
    Input, Conv2D, BatchNormalization, Activation,

```

```

        MaxPooling2D, Add, GlobalAveragePooling2D,
        Dense, Dropout
    )
    from tensorflow.keras.models import Model
    from tensorflow.keras.preprocessing.image import ImageDataGenerator
    from tensorflow.keras.callbacks import (
        EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
    )
    import json, os
    # -----
    # Dataset & Augmentation
    # -----
    dataset_path = r"D:\FYP\character_recognition\BalancedDataset\balanced_train"
    datagen = ImageDataGenerator(
        rescale=1./255,
        rotation_range=10,
        width_shift_range=0.10,
        height_shift_range=0.10,
        zoom_range=0.10,
        shear_range=0.05,
        horizontal_flip=True,
        validation_split=0.2
    )
    train_gen = datagen.flow_from_directory(
        dataset_path,
        target_size=(224,224),
        color_mode="grayscale",
        class_mode="categorical",
        batch_size=32,
        subset="training",
        shuffle=True
    )
    val_gen = datagen.flow_from_directory(
        dataset_path,
        target_size=(224,224),
        color_mode="grayscale",
        class_mode="categorical",
        batch_size=32,
        subset="validation"
    )
    num_classes = train_gen.num_classes
    # Save class labels
    with open("class_labels.json","w") as f:
        json.dump({v:k for k,v in train_gen.class_indices.items()}, f)
    # -----
    # Residual Block
    # -----
    def residual_block(x, filters):
        shortcut = x

```

```

    x = Conv2D(filters, (3,3), padding='same')(x)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)
    x = Conv2D(filters, (3,3), padding='same')(x)
    x = BatchNormalization()(x)
    if shortcut.shape[-1] != filters:
        shortcut = Conv2D(filters, (1,1))(shortcut)
    x = Add()([x, shortcut])
    x = Activation('relu')(x)
    return x
# -----
# ResNet-Lite Model
# -----
def build_resnet_lite(num_classes):
    inp = Input(shape=(224,224,1))
    x = Conv2D(32, (3,3), padding='same', activation='relu')(inp)
    x = MaxPooling2D((2,2))(x)
    x = residual_block(x, 32)
    x = MaxPooling2D((2,2))(x)
    x = residual_block(x, 64)
    x = MaxPooling2D((2,2))(x)
    x = residual_block(x, 128)
    x = MaxPooling2D((2,2))(x)
    x = residual_block(x, 256)
    x = GlobalAveragePooling2D()(x)
    x = Dense(256, activation="relu")(x)
    x = Dropout(0.4)(x)
    out = Dense(num_classes, activation="softmax")(x)
    return Model(inp, out)
model = build_resnet_lite(num_classes)
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-4),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
callbacks = [
    EarlyStopping(monitor='val_loss', patience=10,
                  restore_best_weights=True),
    ModelCheckpoint("best_resnet_lite.keras", save_best_only=True),
    ReduceLROnPlateau(monitor='val_loss',
                      patience=4, factor=0.3, min_lr=1e-6)
]
history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=50,
    callbacks=callbacks,
    verbose=1
)

```

```

# -----
# Phase 2 { Fine-tuning
# -----

from tensorflow.keras.losses import CategoricalCrossentropy
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_aug = ImageDataGenerator(
    rescale=1./255,
    rotation_range=12,
    width_shift_range=0.10,
    height_shift_range=0.10,
    zoom_range=0.12,
    shear_range=0.05,
    brightness_range=[0.8,1.2],
    validation_split=0.2
)

val_aug = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)

train_gen = train_aug.flow_from_directory(
    dataset_path,
    target_size=(224,224),
    color_mode='grayscale',
    batch_size=32,
    subset='training',
    shuffle=True,
    seed=42
)

val_gen = val_aug.flow_from_directory(
    dataset_path,
    target_size=(224,224),
    color_mode='grayscale',
    batch_size=32,
    subset='validation',
    shuffle=False,
    seed=42
)

# Reload previous best model
model = tf.keras.models.load_model("best_resnet_lite.keras")
loss = CategoricalCrossentropy(label_smoothing=0.05)
opt = Adam(learning_rate=5e-5)
model.compile(optimizer=opt, loss=loss, metrics=["accuracy"])
callbacks = [
    EarlyStopping(monitor='val_loss', patience=12,
                  restore_best_weights=True),
    ModelCheckpoint("best_resnet_lite_v2.keras",
                   save_best_only=True,
                   monitor="val_accuracy"),

```

```
        ReduceLROnPlateau(monitor='val_loss',
                           patience=5, factor=0.3, min_lr=1e-7)
]
history2 = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=40,
    callbacks=callbacks,
    verbose=1
)
model.save("resnet_lite_iter2_final.keras")
```

B.1.2 Django Backend Code

The Django backend loads the trained models and performs inference on incoming prescription images.

```
# ml_model.py
import os
import json
import numpy as np
from PIL import Image
from django.conf import settings
from tensorflow.keras.models import load_model
import base64
import io

# ---- Directories ----
BASE_DIR = settings.BASE_DIR
MODEL_DIR = os.path.join(BASE_DIR, "model")
LABELS_PATH = os.path.join(MODEL_DIR, "class_labels.json")
MODELS = {
    "b0": load_model(os.path.join(MODEL_DIR, "EfficientB0.keras")),
    "b1": load_model(os.path.join(MODEL_DIR, "EfficientB1.keras")),
    "b2": load_model(os.path.join(MODEL_DIR, "EfficientB2.keras")),
    "b3": load_model(os.path.join(MODEL_DIR, "resnet_lite_iter4_final.keras")),
}

DEFAULT_MODEL = MODELS["b0"]

# ---- Load Labels ----
with open(LABELS_PATH, "r") as f:
    CLASS_LABELS = json.load(f)

IMG_SIZE = (224, 224)
COLOR_MODE = "L" # grayscale

def prepare_for_model(image: Image.Image):
    """Convert image to (224,224,1) model input format."""
    img = image.convert(COLOR_MODE)
    img = img.resize(IMG_SIZE)
    img_array = np.array(img, dtype=np.float32) / 255.0
    img_array = np.expand_dims(img_array, axis=-1) # channel
    img_array = np.expand_dims(img_array, axis=0) # batch
    return img_array

def run_prediction(images, model_type="b0"):
    model = MODELS.get(model_type, DEFAULT_MODEL)
    results = []
    for img in images:
        if isinstance(img, Image.Image):
            pil_img = img
        elif isinstance(img, str):
            try:
                img_bytes = base64.b64decode(img)
            except:
                img_bytes = base64.b64decode(img.split(",")[-1])
            pil_img = Image.open(io.BytesIO(img_bytes))
```

```
elif hasattr(img, "shape"):
    pil_img = Image.fromarray(img)
else:
    raise ValueError(f"Unsupported image type: {type(img)}")
img_array = prepare_for_model(pil_img)
pred = model.predict(img_array)
predicted_class = int(np.argmax(pred))
confidence = float(np.max(pred))
label = CLASS_LABELS.get(str(predicted_class), str(predicted_class))
results.append({
    "prediction_label": label,
    "confidence": confidence
})
return {"results": results}
```

CHAPTER C

APPENDIX C: Alignment with Sustainable Development Goals (SDGs)

Table C.1: Alignment of the Proposed System with Sustainable Development Goals (SDGs).

No.	SDG Name	Goal	Implementation	Impact
1	Good Health and Well-being (SDG 3)	Reduce medication errors and prevent adverse drug events caused by illegible handwriting.	The system utilizes deep learning models (Efficient-Net B0,B1,B2,ResNet) to accurately interpret pharmaceutical terms, ensuring the correct drug and dosage are dispensed.	Enhances patient safety by minimizing interpretation errors and preventing treatment delays often caused by manual clarification of prescriptions.
2	Quality Education (SDG 4)	Facilitate technical training and knowledge sharing in medical informatics.	The project serves as an educational tool for pharmacy students and researchers, demonstrating practical applications of AI in healthcare and handling diverse handwriting styles.	Promotes skill development in AI and OCR technologies at the undergraduate level and fosters research in medical image processing.
3	Decent Work and Economic Growth (SDG 8)	Improve workflow efficiency and reduce manual workload for healthcare professionals.	By automating the transcription of handwritten notes, the system reduces the time pharmacists spend on manual data entry and verification.	Streamlines pharmacy operations, allowing healthcare workers to focus on patient consultation rather than administrative tasks.
4	Industry, Innovation, and Infrastructure (SDG 9)	Drive technological innovation in healthcare IT using advanced AI architectures.	Implementation of a custom "pipeline-based" OCR system (Segmentation + Recognition) that outperforms generic tools like Tesseract in medical contexts.	Modernizes healthcare infrastructure by enabling digital transformation in regions that still rely on manual paper workflows.
5	Reduced Inequalities (SDG 10)	Bridge the digital gap in developing nations with limited Electronic Health Record (EHR) adoption.	The application is designed to be lightweight and deployable on standard hardware (non-high-end GPUs), making it accessible in low-resource settings.	Democratizes access to advanced medical technology, ensuring that clinics in developing regions can benefit from digital accuracy.
6	Sustainable Cities and Communities (SDG 11)	Promote sustainable healthcare documentation and reduce paper usage.	Digitization of physical prescriptions creates searchable, organized electronic records, reducing the need for physical storage space.	Contributes to eco-friendly "paperless" hospital initiatives and ensures the long-term preservation of patient medical history.
7	Partnerships for the Goals (SDG 17)	Enhance interoperability and collaboration between healthcare systems.	The system is designed with export capabilities (JSON, CSV) to integrate seamlessly with existing Hospital Management Systems and EHR databases.	Facilitates data exchange between doctors and pharmacies, supporting a unified and connected healthcare ecosystem.